

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA ĐIỆN ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH



HCMUTE

ĐỒ ÁN TỐT NGHIỆP

XE ROBOT TỰ HÀNH ĐỊNH VỊ BẰNG

ODOMETRY VÀ TÌM ĐƯỜNG A*

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

Sinh viên: NGUYỄN QUỐC VIỆT

MSSV: 22119254

VÕ VĂN TRỰC

MSSV: 22119251

TP. HỒ CHÍ MINH - 6/2026

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH

KHOA ĐIỆN ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH

ĐỒ ÁN TỐT NGHIỆP

XE ROBOT TỰ HÀNH ĐỊNH VỊ BẰNG ODOMETRY VÀ TÌM ĐƯỜNG A*

Ngành Công Nghệ Kỹ Thuật Máy tính

Sinh viên: NGUYỄN QUỐC VIỆT

MSSV: 22119254

VÕ VĂN TRỰC

MSSV: 22119251

Hướng dẫn: ThS. ĐẶNG PHƯỚC HẢI TRANG

TP. HỒ CHÍ MINH - 6/2026

LỜI CẢM ƠN

Để hoàn thành tốt báo cáo đồ án tốt nghiệp của ngành Công nghệ Kỹ thuật Máy tính, trước hết chúng tôi xin chân thành cảm ơn đến quý Thầy/Cô trong trường Đại học Sư phạm Kỹ thuật Thành phố Hồ Chí Minh nói chung và khoa Điện -Điện tử nói riêng. Đặc biệt là thầy ThS. Đặng Phước Hải Trang đã tận tình hướng dẫn, giúp đỡ và tạo điều kiện thuận lợi cho chúng tôi trong suốt quá trình thực hiện đồ án. Chúng tôi xin gửi đến thầy lời cảm ơn chân thành nhất.

Đồng thời chúng tôi cũng xin gửi lời cảm ơn đến bạn bè vì đã hỗ trợ, đóng góp ý kiến trong quá trình thực hiện đề tài.

Mặc dù đã cố gắng hết sức nhưng vì kiến thức còn hạn chế, không thể tránh khỏi những thiếu sót. Do đó, chúng tôi rất mong nhận được sự đóng góp ý kiến từ phía Thầy/Cô để có thể hoàn thiện tốt hơn nữa, đồng thời tích lũy thêm kinh nghiệm để đi vào thực tế sau này.

Cuối cùng, chúng tôi xin chúc Thầy/Cô luôn có thật nhiều sức khỏe, luôn tràn đầy nhiệt huyết và thành công hơn nữa trong quá trình giảng dạy.

TÓM TẮT

Trong thời đại công nghiệp 4.0, robot di động tự hành (Autonomous Mobile Robot) ngày càng được ứng dụng rộng rãi trong kho vận, dịch vụ và nghiên cứu. Nhằm củng cố kiến thức về hệ thống nhúng, điều khiển và lập trình ứng dụng, nhóm thực hiện đã xây dựng một xe robot tự hành nhỏ sử dụng vi điều khiển ESP32 làm trung tâm điều khiển, kết hợp với một ứng dụng di động Flutter làm giao diện điều hành.

Hệ thống cho phép người dùng chạm chọn điểm đích trên bản đồ; ứng dụng dùng thuật toán A* để tính đường đi ngắn nhất trên đồ thị waypoint, rồi gửi danh sách điểm trung gian xuống ESP32 giao thức UDP trên mạng WiFi do chính ESP32 phát ra. ESP32 bám theo lộ trình bằng cách kết hợp odometry từ encoder bánh xe và cảm biến góc MPU-6050 (bộ lọc bù - complementary filter) để ước lượng vị trí và hướng theo thời gian thực, điều khiển bốn động cơ DC qua mạch cầu L298N. Vị trí xe được phản hồi liên tục về ứng dụng (telemetry 10 Hz) để hiển thị trên bản đồ. Hệ thống còn tích hợp cơ chế tự hiệu chuẩn sai số động cơ/encoder lưu trên bộ nhớ NVS, và chế độ giả lập (simulator) để kiểm thử ứng dụng khi chưa có phần cứng.

Bên cạnh chế độ bám lộ trình theo bản đồ, hệ thống còn có chế độ tự hành né vật cản (autonomous exploration): xe tự chạy thẳng, dùng cảm biến siêu âm HC-SR04 gắn trên một servo quét để liên tục đo khoảng cách phía trước; khi gặp vật cản gần (dưới 30 cm) xe dừng lại, quay servo đo khoảng cách hai bên trái/phải rồi tự quyết định hướng đi thông thoáng hơn (rẽ 90° về bên rộng hơn, hoặc quay đầu 180° khi cả hai phía đều bị chặn, hoặc dừng hẳn khi bị kẹt). Người dùng bật/tắt chế độ này bằng một nút trên ứng dụng.

Thiết kế module hóa, chi phí thấp, không phụ thuộc Internet (mạng cục bộ), phù hợp làm nền tảng học thuật cho các bài toán định vị và dẫn đường robot trong quy mô phòng thí nghiệm.

MỤC LỤC

LỜI CẢM ƠN	iii
TÓM TẮT	iv
MỤC LỤC.....	v
DANH MỤC HÌNH.....	ix
DANH MỤC BẢNG	xi
DANH SÁCH CÔNG THỨC	xii
CÁC TỪ VIẾT TẮT	1
CHƯƠNG 1: TỔNG QUAN.....	3
1.1 GIỚI THIỆU	3
1.2 MỤC TIÊU ĐỀ TÀI	5
1.3 ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU.....	6
1.4 PHƯƠNG PHÁP NGHIÊN CỨU.....	7
1.5 BỐ CỤC ĐỀ TÀI.....	7
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	9
2.1 CÁC GIAO THỨC KẾT NỐI SỬ DỤNG	9
2.1.1 GPIO	9
2.1.2 I2C.....	9
2.1.3 PWM và bộ LEDC của ESP32	10
2.1.3 Trigger - Echo (cảm biến siêu âm)	11
2.1.4 Ngắt ngoài và đọc encoder.....	12
2.1.5 WiFi và giao thức UDP.....	12
2.2 CÁC THUẬT TOÁN VÀ LÝ THUYẾT NỀN TẢNG	13
2.2.1 Mô hình xe vi sai và odometry	13

2.2.2 Bộ lọc bù (Complementary Filter).....	13
2.2.3 Thuật toán tìm đường A*	14
2.3 CÁC PHẦN MỀM XÂY DỰNG VÀ HỖ TRỢ	15
2.3.1 PlatformIO trên Visual Studio Code.....	15
2.3.2 Flutter và ngôn ngữ Dart.....	16
2.3.3 Kicad (thiết kế mạch).....	16
2.4 CÁC THƯ VIỆN SỬ DỤNG.....	16
CHƯƠNG 3: THIẾT KẾ VÀ THI CÔNG	18
3.1 YÊU CẦU VÀ SƠ ĐỒ KHỐI VÀ HỆ THỐNG	18
3.1.1 Yêu cầu của hệ thống	18
3.1.2 Sơ đồ khối và chức năng mỗi khối	18
3.1.3 Chức năng của từng khối	19
3.1.4 Các hoạt động của hệ thống	20
3.2 THIẾT KẾ PHẦN CỨNG	20
3.2.1 Khối xử lý trung tâm (ESP32)	21
3.2.2 Khối động cơ (mạch lái L298N + 4 động cơ DC)	22
3.2.3 Khối encoder.....	25
3.2.4 Khối cảm biến góc IMU (MPU6050)	26
3.2.5 Khối cảm biến khoảng cách (HC-SR04)	27
3.2.6 Khối servo quét cảm biến	29
3.2.7 Khối cảnh báo (buzzer).....	31
3.2.8 Khối nguồn.....	31
3.3 THIẾT KẾ PHẦN MỀM ỨNG DỤNG DI ĐỘNG (FLUTTER).....	31
3.4 GIAO THỨC TRUYỀN THÔNG UDP	35
3.5 SƠ ĐỒ NỐI DÂY HỆ THỐNG.....	36

3.6 LƯU ĐỒ GIẢI THUẬT	38
3.6.1 Lưu đồ giải thuật chính (firmware).....	38
3.6.2 Lưu đồ chương trình con cập nhật Odometry	39
3.6.3 Lưu đồ điều hướng node - cơ chế 3 tầng xoay	41
3.6.4 Lưu đồ chương trình con đi thẳng (heading correction)	42
3.6.5 Lưu đồ luồng điều hướng trên ứng dụng (A*).....	44
3.6.6 Lưu đồ giải thuật chế độ tự hành né vật cản	45
3.7 DANH SÁCH LINH KIỆN	47
3.8 THI CÔNG PHẦN CỨNG VÀ NỐI DÂY.....	47
3.9 LẬP TRÌNH FIRMWARE VÀ ỨNG DỤNG	49
CHƯƠNG 4: KẾT QUẢ VÀ THẢO LUẬN.....	51
4.1 KẾT QUẢ LÝ THUYẾT.....	51
4.2 KẾT QUẢ THỰC HÀNH.....	51
4.2.1 Điều kiện và phương pháp thực nghiệm.....	52
4.2.2 Độ chính xác đo quãng đường (odometry)	52
4.2.3 Độ chính xác quay góc IMU.....	52
4.2.4 Khả năng giữ hướng khi đi thẳng	53
4.2.5 Độ chính xác điều hướng theo lưới điểm (node)	54
4.2.6 Độ chính xác cảm biến siêu âm kết hợp servo quét.....	54
4.2.7 Khả năng né vật cản ở chế độ tự hành	55
4.2.7 Độ trôi góc hướng của IMU (drift)	55
4.3 NHẬN XÉT VÀ ĐÁNH GIÁ.....	56
4.3.1 Đánh giá mức độ đạt mục tiêu	57
4.3.2 Đánh giá chung	58
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	60

5.1 KẾT LUẬN.....	60
5.2 HƯỚNG PHÁT TRIỂN.....	60
PHỤ LỤC.....	62
TÀI LIỆU THAM KHẢO	63

DANH MỤC HÌNH

Hình 2.1: Mô hình giao tiếp I2C đã thiết bị.....	10
Hình 2.2 Minh họa các mức duty cycle của tín hiệu PWM.....	11
Hình 2.3: Nguyên lý đo khoảng cách bằng cảm biến siêu âm HC-SR04.....	12
Hình 2.4: So sánh kết quả lập kế hoạch đường đi A trước và sau khi tối ưu góc rẽ.....	15
Hình 2.5: Giao diện PlatformIO trên VS Code.....	16
Hình 3.1: Sơ đồ khối hệ thống.....	19
Hình 3.2 Sơ đồ chân của ESP32-WROOM-32.....	22
Hình 3.3 Sơ đồ kết nối ESP32 - L298N – bốn động cơ DC (nhóm trái/phải).....	24
Hình 3.4: Sơ đồ kết nối encoder LM393 với ESP32	25
Hình 3.5 Sơ đồ kết nối MPU-6050 qua I2C.....	27
Hình 3.6 Sơ đồ kết nối HC-SR04 với ESP32.....	28
Hình 3.7: Sơ đồ kết nối servo quét cảm biến với ESP32	30
Hình 3.8: Bản đồ lưới waypoint.....	33
Hình 3.9: Các giao diện chính của ứng dụng điều khiển xe.....	34
Hình 3.10: Sơ đồ nguyên lý toàn hệ thống.....	37
Hình 3.11: Lưu đồ giải thuật chính.....	39
Hình 3.12: Lưu đồ cập nhật odometry	40
Hình 3.13: Lưu đồ điều hướng node 3 tăng xoay	41
Hình 3.14: Lưu đồ chương trình con đi thẳng	43
Hình 3.15: Lưu đồ luồng điều hướng trên app	44
Hình 3.16: Lưu đồ giải thuật chế độ tự hành né vật cản	46
Hình 3.17 Ảnh xe robot sau khi lắp ráp hoàn chỉnh	49

<i>Hình 3.18: Giao diện bản đồ trong quá trình xe di chuyển</i>	50
---	----

DANH MỤC BẢNG

<i>Bảng 2.1: Các thư viện dùng trong firmware</i>	<i>16</i>
<i>Bảng 2.2: Các thư viện dùng trong App Flutter</i>	<i>17</i>
<i>Bảng 3.1 Bảng phân công chân GPIO cho các thiết bị ngoại vi</i>	<i>20</i>
<i>Bảng 3.2 Thông số kỹ thuật của module L298N</i>	<i>22</i>
<i>Bảng 3.3: Thông số servo SG90 và tham số chế độ tự hành</i>	<i>29</i>
<i>Bảng 3.4: Các giá trị nguồn của thiết bị.....</i>	<i>31</i>
<i>Bảng 3.5: Phân bố cổng và chức năng truyền thông</i>	<i>35</i>
<i>Bảng 3.6: Các giá trị trạng thái (status) trong telemetry</i>	<i>36</i>
<i>Bảng 3.7: Các tham số điều khiển chính của firmware</i>	<i>38</i>
<i>Bảng 3.8 : Bảng liệt kê các linh kiện của hệ thống.....</i>	<i>47</i>
<i>Bảng 4.1 Đánh giá độ chính xác ước lượng quãng đường khi xe di chuyển thẳng.....</i>	<i>52</i>
<i>Bảng 4.2 Kết quả đo độ chính xác xoay góc</i>	<i>53</i>
<i>Bảng 4.3 Kết quả đánh giá khả năng giữ hướng</i>	<i>53</i>
<i>Bảng 4.4 Kết quả điều hướng theo lộ trình node.....</i>	<i>54</i>
<i>Bảng 4.5 Kết quả đo khoảng cách bằng HC-SR04 theo ba hướng quét.....</i>	<i>54</i>
<i>Bảng 4.6 Kết quả thử nghiệm né vật cản tự hành</i>	<i>55</i>
<i>Bảng 4.7 Kết quả đo của độ trôi góc yaw khi đứng yên</i>	<i>55</i>
<i>Bảng 4.8: Bảng đánh giá hoạt động của hệ thống.....</i>	<i>56</i>
<i>Bảng 4.9 Đánh giá mức độ hoàn thành so với mục tiêu đề ra.....</i>	<i>58</i>

DANH SÁCH CÔNG THỨC

Công thức	Nội dung
Công thức 2.1	Khoảng cách từ cảm biến siêu âm
Công thức 2.2	Quãng đường tịnh tiến trung tâm xe
Công thức 2.3	Góc quay của xe
Công thức 2.4	Cập nhật vị trí theo hệ trục x
Công thức 2.5	Cập nhật vị trí theo hệ trục y
Công thức 2.6	Bộ lọc bù kết hợp hai nguồn để bù khuyết điểm của nhau
Công thức 2.7	Hàm chi phí và heuristic của A*
Công thức 3.1	Chu vi và quãng đường mỗi xung encoder
Công thức 3.2	Chu vi bánh xe
Công thức 3.3	Chuyển đổi gyro thô sang vận tốc góc
Công thức 3.4	Tích phân yaw từ con quay hồi chuyển
Công thức 3.5	Quy đổi góc servo sang độ rộng xung
Công thức 3.6	Quy đổi góc servo sang duty LEDC
Công thức 3.7	Góc quay vi sai từ encoder
Công thức 3.8	Bộ lọc bù hợp nhất hướng
Công thức 3.9	Cập nhật tọa độ (x, y)
Công thức 3.10	Số xung mục tiêu cho một đoạn đường
Công thức 3.11	Hiệu chỉnh hướng (heading correction) khi đi thẳng
Công thức 3.12	Cập nhật hệ số bù động cơ trái
Công thức 3.13	Cập nhật hệ số bù động cơ phải
Công thức 3.14	Tính sai lệch encoder chuẩn hóa

CÁC TỪ VIẾT TẮT

Viết tắt	Tiếng Anh	Nghĩa tiếng Việt
ESP32		Vi điều khiển ESP32 (lõi kép, tích hợp WiFi/BLE)
GPIO	General Purpose Input/Output	Chân vào/ra tổng quát
I2C	Inter-Integrated Circuit	Giao tiếp giữa các mạch tích hợp
UART	Universal Asynchronous Receiver/Transmitter	Bộ thu/phát không đồng bộ
SPI	Serial Peripheral Interface	Giao tiếp ngoại vi nối tiếp
PWM	Pulse Width Modulation	Điều chế độ rộng xung
LEDC	LED Control (ESP32 PWM peripheral)	Bộ tạo PWM phần cứng của ESP32
PID	Proportional-Integral-Derivative	Bộ điều khiển tỉ lệ - tích phân - vi phân
IMU	Inertial Measurement Unit	Cảm biến quán tính (gia tốc + con quay)
MPU	Motion Processing Unit	Vi mạch cảm biến chuyển động (MPU-6050)
DLPF	Digital Low-Pass Filter	Bộ lọc thông thấp số (trong MPU-6050)
LSB	Least Significant Bit	Bit có trọng số thấp nhất (đơn vị giá trị thô)
dps	degrees per second	Độ trên giây (đơn vị vận tốc góc)
UDP	User Datagram Protocol	Giao thức truyền không kết nối
JSON	JavaScript Object Notation	Định dạng trao đổi dữ liệu dạng văn bản

AP / SoftAP	(Software) Access Point	Điểm truy cập WiFi phần mềm
SSID	Service Set Identifier	Tên mạng WiFi
IP	Internet Protocol (address)	Địa chỉ giao thức Internet
NVS	Non-Volatile Storage	Bộ nhớ không mất dữ liệu (flash của ESP32)
ISR	Interrupt Service Routine	Chương trình phục vụ ngắt
A*	A-star algorithm	Thuật toán tìm đường A-sao
DC	Direct Current	Dòng điện một chiều (động cơ DC)
RPM	Revolutions Per Minute	Số vòng quay mỗi phút
ToF	Time of Flight	Cảm biến đo thời gian bay (laser)
PCB	Printed Circuit Board	Bảng mạch in
IDE	Integrated Development Environment	Môi trường phát triển tích hợp

CHƯƠNG 1: TỔNG QUAN

1.1 GIỚI THIỆU

Robot di động tự hành (Autonomous Mobile Robot – AMR) đang trở thành một trong những hướng công nghệ phát triển nhanh nhất, gắn liền với làn sóng tự động hóa và cuộc Cách mạng công nghiệp 4.0. Khác với robot cố định hay xe dẫn đường tự động (AGV) truyền thống vốn bám theo vạch từ/băng dẫn cố định, AMR có khả năng tự định vị, tự lập kế hoạch đường đi và tự điều hướng trong môi trường, nhờ đó được ứng dụng ngày càng rộng rãi trong kho vận thông minh (intralogistics), robot dịch vụ, y tế, nông nghiệp và các nền tảng nghiên cứu [1], [2]. Nhiều khảo sát cho thấy việc thay thế AGV cứng nhắc bằng AMR linh hoạt giúp tăng đáng kể năng suất và khả năng thích ứng của dây chuyền sản xuất trong mô hình nhà máy thông minh. [2]

Dù đa dạng về hình thức, mọi robot tự hành đều phải giải quyết ba bài toán cốt lõi: định vị (robot biết mình đang ở đâu), lập kế hoạch đường đi (xác định lộ trình tới đích) và điều khiển bám lộ trình (di chuyển chính xác theo lộ trình đó) [1]. Về định vị, phương pháp tương đối (dead-reckoning) dựa trên odometry từ encoder bánh xe đơn giản, rẻ và tần số cao nhưng sai số tích lũy (trôi) theo quãng đường do trượt bánh và đường kính bánh không đều [3]; phương pháp tuyệt đối dùng GPS hoặc mốc định vị (landmark/beacon), còn các kỹ thuật hiện đại như định vị và dựng bản đồ đồng thời (SLAM) bằng LiDAR/camera cho độ chính xác cao nhưng đòi hỏi cảm biến đắt tiền và năng lực tính toán lớn [4].

Để giảm trôi của odometry, nhiều nghiên cứu đã hợp nhất encoder với cảm biến quán tính (IMU). [5] hợp nhất nhiều cảm biến (odometry, IMU) bằng bộ lọc Kalman mở rộng (EKF), nâng cao rõ rệt độ chính xác định vị và dẫn đường so với chỉ dùng đơn cảm biến. Cùng hướng đó, Ly và cộng sự [6] hợp nhất dữ liệu IMU và encoder bằng bộ lọc Kalman cho robot di động, cho thấy vị trí ước lượng bám quỹ đạo thực sát hơn hẳn odometry thuần; Housein và cộng sự [7] trình bày một triển khai EKF thực nghiệm cho cùng bài toán. Các hướng mở rộng gần đây còn kết hợp thêm thị giác (wheel–inertial–visual) bằng biến thể bộ lọc Kalman để tăng

độ bền vững [8]. Tuy nhiên, bộ lọc Kalman/EKF đòi hỏi mô hình và khối lượng tính toán đáng kể; với vi điều khiển hạn chế tài nguyên, bộ lọc bù (complementary filter) là giải pháp nhẹ hơn, đủ tốt để kết hợp con quay hồi chuyển và encoder nhằm ổn định ước lượng hướng [9].

Về bài toán tìm đường, lý thuyết đồ thị cung cấp nền tảng vững chắc: thuật toán Dijkstra tìm đường đi ngắn nhất trên đồ thị có trọng số [10], còn thuật toán A mở rộng Dijkstra bằng một hàm heuristic để hướng việc tìm kiếm về phía đích, nhờ đó giảm mạnh số nút phải duyệt mà vẫn đảm bảo tối ưu khi heuristic chấp nhận được (admissible) [11]. Bên cạnh nhóm dựa trên đồ thị/lưới còn có nhóm lấy mẫu ngẫu nhiên (RRT, PRM) cho không gian cấu hình nhiều chiều [12]. Trong thực tiễn robot, A vẫn là lựa chọn phổ biến và liên tục được cải tiến: Shahapur và cộng sự [13] hiện thực và so sánh A với Dijkstra trên robot TurtleBot3 (mô phỏng Gazebo), xác nhận A tìm được đường tối ưu với số nút duyệt ít hơn; Yao và cộng sự [14] đề xuất biến thể A cải tiến (BSGA) cho robot vi sai nhằm khắc phục đúng các hạn chế của A gốc là số nút tìm kiếm lớn và đường đi kém mượt. Với môi trường trong nhà đã biết trước dưới dạng lưới điểm (waypoint), A là lựa chọn cân bằng tốt giữa tính tối ưu, tốc độ thực thi và độ đơn giản khi cài đặt trên thiết bị di động.

Song song với định vị và tìm đường, khả năng né vật cản cũng được nghiên cứu theo hướng chi phí thấp. Yasin và cộng sự [15] đề xuất phương pháp phát hiện vật cản và tránh va chạm chỉ dùng cảm biến siêu âm giá rẻ, với mục tiêu tối thiểu hóa độ lệch khỏi lộ trình ban đầu, rất phù hợp cho robot ngân sách thấp. Ở quy mô công nghiệp, robot tự hành thường được trang bị cảm biến đắt tiền (LiDAR, camera độ phân giải cao, GPS-RTK) cùng máy tính mạnh để chạy SLAM và thị giác máy tính [4]; nhưng chi phí và độ phức tạp đó là rào cản lớn đối với sinh viên và các dự án học thuật. Vì vậy, hướng xây dựng robot tự hành chi phí thấp dựa trên vi điều khiển phổ thông ngày càng được quan tâm, tiêu biểu là ESP32, vi điều khiển lõi kép tích hợp sẵn WiFi/Bluetooth, giá rẻ, cộng đồng hỗ trợ lớn, phù hợp cho các ứng dụng IoT và robot nhỏ [16].

Từ tổng quan trên có thể thấy các kỹ thuật định vị (hợp nhất encoder-IMU),

tìm đường (A) và né vật cản đều đã tương đối hoàn thiện, nhưng phần lớn được kiểm chứng trên nền tảng mạnh (PC/ROS, robot chuẩn như TurtleBot3) [13] hoặc bằng các bộ lọc tính toán nặng như EKF [5] [7]. Vẫn còn thiếu những giải pháp tích hợp trọn vẹn* cả ba chức năng định vị – tìm đường – bám lộ trình lên một vi điều khiển chi phí thấp, hoạt động độc lập không cần Internet và dễ tiếp cận cho mục đích học tập. Xây dựng một nền tảng như vậy vừa mang ý nghĩa thực tiễn, là sản phẩm học tập giúp người thực hiện nắm vững trọn vẹn chuỗi kiến thức nền (điều khiển động cơ, đọc cảm biến, định vị, lập kế hoạch đường đi, truyền thông không dây), vừa là khung sườn để phát triển tiếp các tính năng cao cấp hơn như tránh vật cản động, SLAM hay phối hợp đội hình nhiều robot.

Xuất phát từ những lý do đó, đề tài "Xe robot tự hành định vị bằng odometry và tìm đường A*" được lựa chọn thực hiện. Đề tài dùng ESP32 làm bộ điều khiển trung tâm; định vị bằng odometry kết hợp encoder bánh xe và cảm biến góc MPU-6050 thông qua bộ lọc bù [9]; tìm đường bằng thuật toán A* [11] chạy trên ứng dụng di động đa nền tảng viết bằng Flutter [17]; và liên lạc thời gian thực giữa ứng dụng với xe qua giao thức UDP trên mạng WiFi cục bộ do chính ESP32 phát ra. Việc chọn Flutter và mạng cục bộ giúp hệ thống đa nền tảng, độ trễ thấp và hoạt động độc lập với Internet. Toàn bộ hệ thống được thiết kế theo hướng module hóa, chi phí thấp và có khả năng mở rộng, hướng tới một nền tảng học thuật hoàn chỉnh cho các bài toán định vị và dẫn đường robot ở quy mô phòng thí nghiệm.

1.2 MỤC TIÊU ĐỀ TÀI

Mục tiêu lớn: Thiết kế và xây dựng một xe robot tự hành sử dụng vi điều khiển ESP32, có khả năng tự định vị, tự tìm đường và bám theo lộ trình tới điểm đích do người dùng chọn trên ứng dụng di động.

Mục tiêu, các kết quả dự kiến đạt được:

Về phần cứng:

- Hoàn thiện cơ khí xe robot bốn bánh dẫn động vi sai (skid-steer): khung xe bốn bánh, bốn động cơ DC giảm tốc chia hai bên trái/phải, mỗi bên gắn một encoder quang đo tốc độ.
- Lắp và đấu nối khối điện tử quanh ESP32: mạch lái L298N + 4 động

cơ, IMU MPU-6050 (I2C), HC-SR04 gắn trên servo quét, buzzer; cấp nguồn pin qua module ổn áp.

Về phần mềm:

- Firmware ESP32 module hóa: điều khiển động cơ (PWM/cầu H), đọc encoder bằng ngắt, đọc IMU + siêu âm, ước lượng odometry vi sai kết hợp bộ lọc bù IMU.
- Điều chỉnh hướng node: tự xoay đúng hướng và đi thẳng đúng quãng đường giữa các node, có giữ hướng + cân bằng hai bánh; tự hiệu chuẩn (cân bằng động cơ, tỉ lệ encoder) lưu qua các lần khởi động.
- Né vật cản: chế độ tự hành (siêu âm + servo quét, tự rẽ tránh) và né vật cản động khi bám lộ trình (chặn cạnh trên bản đồ + tính lại đường bằng A*).
- Ứng dụng di động Flutter: hiển thị bản đồ lưới waypoint, tính đường đi ngắn nhất bằng A, gửi lộ trình và hiển thị vị trí xe + trạng thái theo thời gian thực; có chế độ giả lập* chạy không cần phần cứng.
- Giao thức truyền thông UDP độ trễ thấp (lệnh + telemetry) kèm chia mảnh thành các lộ trình.

1.3 ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU

Đối tượng nghiên cứu: hệ thống xe robot tự hành bốn bánh dẫn động vi sai kiểu skid-steer (differential drive) sử dụng ESP32 làm trung tâm điều khiển, định vị bằng odometry encoder kết hợp IMU, dẫn đường bằng A* trên đồ thị waypoint.

Phạm vi nghiên cứu:

- Giới hạn về phần cứng: Hệ thống được triển khai trên mô hình xe robot di động bốn bánh dẫn động vi sai kiểu skid-steer (differential drive), sử dụng vi điều khiển ESP32 làm bộ xử lý trung tâm. Các thành phần phần cứng bao gồm bốn động cơ DC (chia hai bên trái/phải), encoder đo tốc độ bánh xe ở mỗi bên, cảm biến IMU và mạch điều khiển động cơ L298N.
- Giới hạn về phương pháp định vị: Vị trí và hướng di chuyển của robot được ước lượng dựa trên phương pháp odometry encoder kết hợp dữ liệu IMU. Đề tài chỉ tập trung vào định vị tương đối (dead-reckoning), không nghiên

cứu các phương pháp định vị tuyệt đối như GPS hoặc các hệ thống định vị ngoài.

- Giới hạn về thuật toán dẫn đường: Thuật toán tìm đường được giới hạn ở thuật toán A* trên đồ thị waypoint được xây dựng trước. Robot thực hiện di chuyển theo lộ trình đã xác định, chưa xử lý bài toán lập kế hoạch đường đi trong môi trường chưa biết hoặc thay đổi theo thời gian.
- Giới hạn về môi trường thử nghiệm: Hệ thống được thử nghiệm trong môi trường trong nhà với diện tích giới hạn khoảng 5×5 m, mặt sàn phẳng, ít thay đổi về điều kiện hoạt động. Bản đồ môi trường được biết trước và biểu diễn dưới dạng lưới điểm phục vụ quá trình dẫn đường.
- Giới hạn về mô phỏng kịch bản hoạt động: Các kịch bản đánh giá tập trung vào khả năng di chuyển đến vị trí mục tiêu, bám theo quỹ đạo đã lập và đánh giá sai số định vị trong điều kiện môi trường kiểm soát. Đề tài chưa xét đến các trường hợp phức tạp như vật cản động, thay đổi bản đồ hoặc nhiều robot phối hợp.
- Các công nghệ chưa được xem xét: Đề tài không sử dụng các công nghệ định vị và nhận thức môi trường nâng cao như LiDAR, camera AI, GPS hoặc thuật toán SLAM xây dựng bản đồ đồng thời.

1.4 PHƯƠNG PHÁP NGHIÊN CỨU

Đề tài sử dụng kết hợp các phương pháp: phân tích - tổng hợp tài liệu từ các nguồn học thuật và datasheet linh kiện; mô phỏng - thực nghiệm (chế độ simulator của ứng dụng cho phép kiểm thử phần mềm trước khi có phần cứng); tính toán kỹ thuật điện tử (lựa chọn linh kiện, tính toán quang đường/góc quay, hiệu chỉnh sai số); và lập trình nhúng trên ESP32 (C++ với PlatformIO) song song với lập trình ứng dụng (Dart/Flutter). Hệ thống được phát triển theo từng giai đoạn (phase) và kiểm thử đơn vị (unit test) cho các module thuật toán quan trọng.

1.5 BỐ CỤC ĐỀ TÀI

Đề án được chia làm 5 chương như sau:

- Chương 1: Tổng quan:

Giới thiệu chung, nêu tính cấp thiết, xác định mục tiêu, đối tượng, phạm vi

và phương pháp nghiên cứu.

- Chương 2: Cơ sở lý thuyết:

Trình bày các chuẩn giao tiếp (GPIO, I2C, PWM, Trigger-Echo, ngắt, WiFi/UDP), các thuật toán nền tảng (odometry vi sai, bộ lọc bù, A*), các phần mềm và thư viện sử dụng.

- Chương 3: Thiết kế và thi công:

Từ yêu cầu, đưa ra sơ đồ khối, thiết kế phần cứng từng khối, kiến trúc phần mềm ứng dụng, giao thức UDP, sơ đồ nối dây và các lưu đồ giải thuật, Liệt kê linh kiện, lắp ráp phần cứng, nối dây, lập trình firmware và ứng dụng.

- Chương 4: Kết quả và thảo luận:

Trình bày kết quả lý thuyết và thực hành, đánh giá hoạt động của từng khối.

- Chương 5: Kết luận và hướng phát triển

Tổng kết ưu/nhược điểm và đề xuất hướng cải thiện, phát triển hệ thống, hướng đi của hệ thống trong việc phát triển thành sản phẩm thực tế.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 CÁC GIAO THỨC KẾT NỐI SỬ DỤNG

2.1.1 GPIO

GPIO (General Purpose Input/Output) là các chân tín hiệu được cấu hình bằng phần mềm để thực hiện chức năng ngõ vào (input) hoặc ngõ ra (output). Trong đề tài, GPIO của ESP32 được dùng cho hầu hết các kết nối số: điều khiển chiều quay động cơ (các chân IN1-IN4 của L298N), phát xung trigger cho cảm biến siêu âm, điều khiển còi buzzer, và đọc tín hiệu xung từ encoder.

ESP32 sử dụng mức logic 3.3 V [18]. Mỗi chân GPIO có thể cấu hình kéo lên (pull-up)/kéo xuống (pull-down) nội bộ, và quan trọng nhất với đề tài là khả năng tạo ngắt ngoài (interrupt) khi có cạnh tín hiệu - được dùng để đếm xung encoder. Cần lưu ý một số chân của ESP32 chỉ có chức năng input (GPIO34, GPIO35, GPIO36, GPIO39 - không có điện trở kéo nội bộ); trong đề tài chân GPIO34 và GPIO32 được chọn làm ngõ vào đọc encoder.

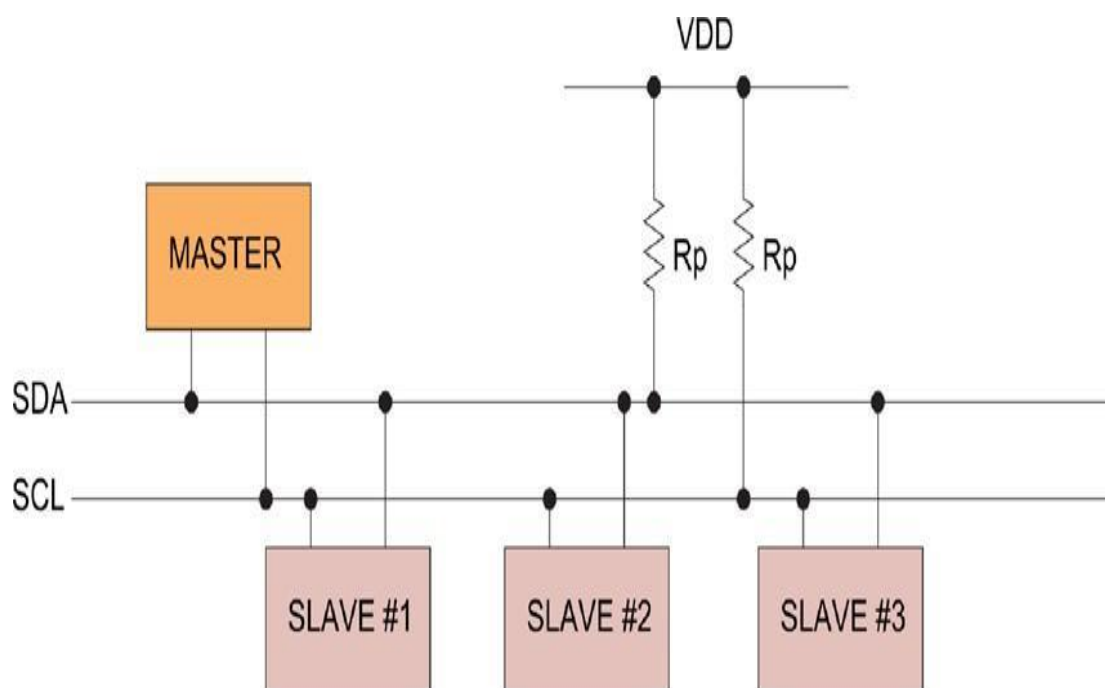
2.1.2 I2C

I2C (Inter-Integrated Circuit) [19] là giao thức truyền thông nối tiếp đồng bộ, sử dụng hai đường:

SDA (Serial Data Line): đường dữ liệu hai chiều.

SCL (Serial Clock Line): đường xung nhịp do master cung cấp.

Hệ thống I2C gồm một master (ESP32) điều khiển bus và một hoặc nhiều slave được gán địa chỉ. Trong đề tài, I2C dùng để giao tiếp với cảm biến quán tính MPU-6050 (địa chỉ mặc định 0x68, hoặc 0x69 khi chân AD0 ở mức cao). ESP32 đặt chân GPIO21 = SDA, GPIO22 = SCL và chạy bus ở tốc độ 50 kHz để đảm bảo ổn định khi đi dây dài và có nhiễu từ động cơ.



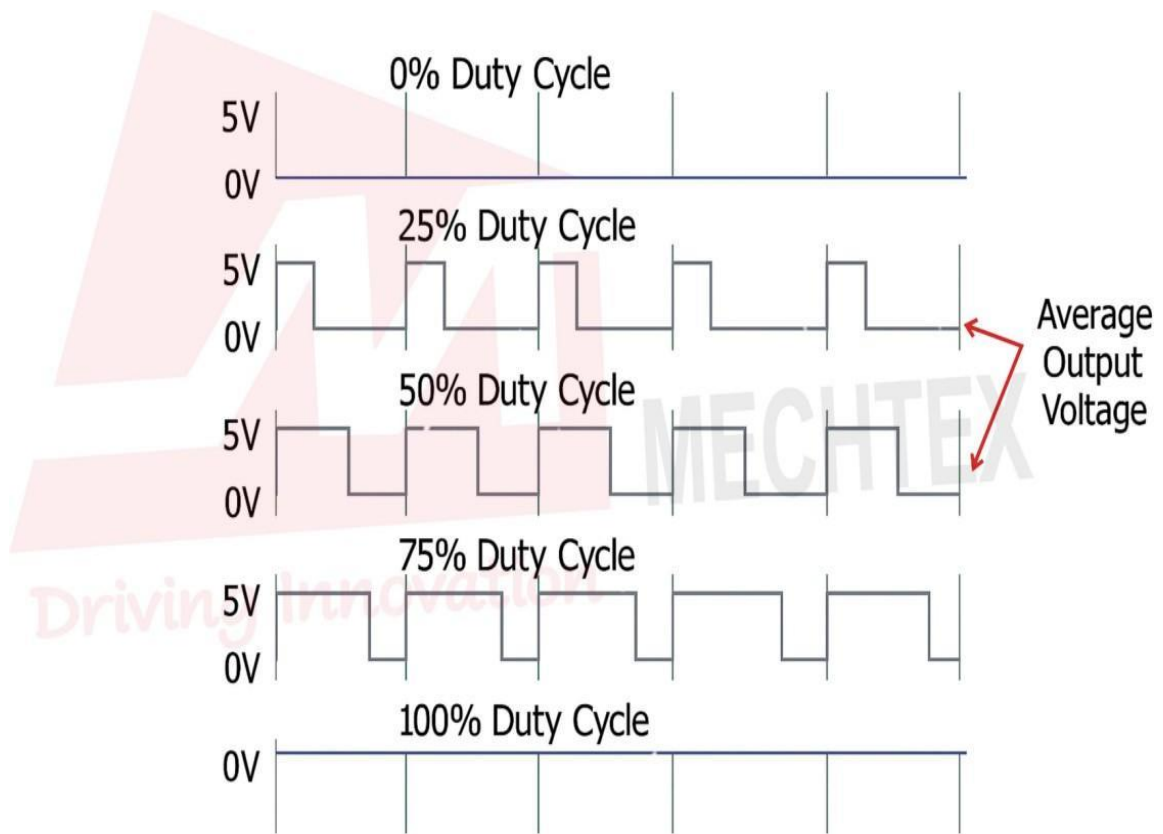
Hình 2.1: Mô hình giao tiếp I2C đa thiết bị

2.1.3 PWM và bộ LEDC của ESP32

PWM (Pulse Width Modulation – điều chế độ rộng xung) [20] điều khiển công suất trung bình cấp cho tải bằng cách thay đổi tỉ lệ thời gian mức cao/thấp của tín hiệu (duty cycle). Tốc độ động cơ DC tỉ lệ với duty cycle của tín hiệu PWM cấp vào chân Enable của mạch cầu H.

ESP32 có bộ ngoại vi LEDC [21] chuyên tạo PWM phần cứng (không tốn CPU). Trong đề tài, hai kênh LEDC (kênh 0 cho bánh trái, kênh 1 cho bánh phải) được cấu hình tần số 5 kHz, độ phân giải 8-bit (duty 0–255), gắn vào hai chân Enable ENA/ENB của L298N. Giá trị PWM 0 là dừng, 255 là tốc độ tối đa.

Bộ LEDC còn được dùng để điều khiển servo quét cảm biến: một kênh LEDC riêng (kênh 2) được cấu hình tần số 50 Hz, độ phân giải 16-bit - đúng chuẩn tín hiệu servo RC (chu kỳ 20 ms, độ rộng xung 0,5–2,5 ms tương ứng góc 0–180°). Việc dùng kênh và bộ định thời (timer) tách biệt với hai kênh động cơ giúp servo và motor hoạt động đồng thời mà không xung đột tần số.



Hình 2.2 Minh họa các mức duty cycle của tín hiệu PWM

2.1.3 Trigger - Echo (cảm biến siêu âm)

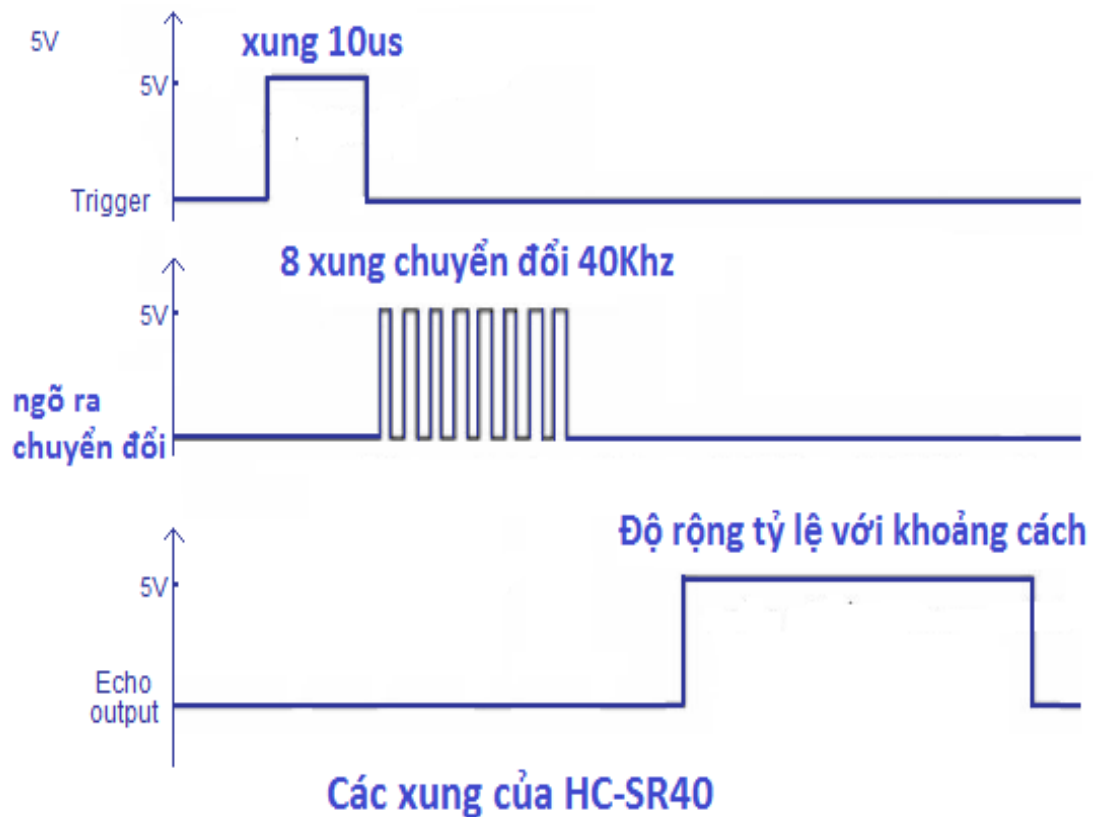
Cảm biến siêu âm HC-SR04 đo khoảng cách theo nguyên lý phản hồi sóng âm với hai chân:

Trigger: ngõ vào, vi điều khiển phát một xung mức cao 10 μ s để bắt đầu phép đo.

Echo: ngõ ra, trả về một xung có độ rộng tỉ lệ với thời gian sóng âm đi tới vật cản và phản xạ về.

ESP32 đo độ rộng xung Echo (bằng hàm) rồi suy ra khoảng cách. Vận tốc âm thanh trong không khí xấp xỉ 343 m/s (0,0343 cm/ μ s); vì sóng đi và về nên quãng đường tính được phải chia 2:

$$Distance(cm) = \frac{t_{echo}(\mu s) \times 0.0343}{2} = \frac{t_{echo}(\mu s)}{58.3} \quad (2.1)$$



Hình 2.3: Nguyên lý đo khoảng cách bằng cảm biến siêu âm HC-SR04

2.1.4 Ngắt ngoài và đọc encoder

Encoder bánh xe tạo ra một chuỗi xung khi bánh quay. Để không bỏ sót xung khi vi điều khiển đang bận, đề tài dùng ngắt ngoài (external interrupt): mỗi cạnh lên (RISING) trên chân encoder kích hoạt một ISR (Interrupt Service Routine) tăng bộ đếm xung. ISR được đặt trong

RAM nội (IRAM_ATTR) để thực thi nhanh.

Do encoder quang LM393 một kênh (chỉ có ngõ D0) không cho biết chiều quay, firmware lấy chiều quay từ lệnh điều khiển động cơ (motor đang tiến thì xung cộng, lùi thì xung trừ). Để lọc nhiễu điện từ (EMI) do động cơ gây ra, ISR áp dụng chống dội phần mềm: bỏ qua các xung cách nhau dưới 6000 μs. Ngưỡng này còn lọc các cạnh giả do comparator LM393 (thiếu hysteresis) [22] sinh ra ở mép khe đĩa.

2.1.5 WiFi và giao thức UDP

ESP32 tích hợp WiFi [23], có thể hoạt động ở chế độ Access Point (SoftAP) - tự phát ra một mạng WiFi để điện thoại kết nối trực tiếp, không cần router. Đề tài

dùng chế độ này để tạo mạng cục bộ độ trễ thấp, hoạt động độc lập không cần Internet.

Trên mạng đó, dữ liệu được trao đổi bằng UDP (User Datagram Protocol) [24] - giao thức không kết nối, không bắt tay, không đảm bảo thứ tự/độ tin cậy nhưng độ trễ rất thấp, rất phù hợp cho điều khiển robot thời gian thực: telemetry vị trí gửi 10 lần/giây có thể chấp nhận mất vài gói, còn lệnh quan trọng (lộ trình) được gửi lặp lại nhiều lần để bù.

2.2 CÁC THUẬT TOÁN VÀ LÝ THUYẾT NỀN TẢNG

2.2.1 Mô hình xe vi sai và odometry

Xe robot trong đề tài là loại bốn bánh vi sai (differential drive) [25], [26]: bốn bánh chủ động trái/phải điều khiển độc lập, hướng xe thay đổi nhờ chênh lệch tốc độ bốn bánh. Odometry (đo hành trình) [27], [28] là phương pháp ước lượng vị trí robot bằng cách tích lũy quãng đường mỗi bánh đo được từ encoder.

Gọi d_L , d_R là quãng đường bánh trái/phải đi được trong một chu kỳ, L là khoảng cách giữa hai bánh (wheelbase). Khi đó:

$$\text{Quãng đường tịnh tiến trung tâm xe: } d = \frac{d_L + d_R}{2} \quad (2.2)$$

$$\text{Góc quay của xe: } \Delta\theta = \frac{d_R - d_L}{L} \quad (2.3)$$

Cập nhật vị trí theo hệ trục (lưu ý hệ tọa độ của bản đồ có trục Y hướng xuống):

$$x += d \times \cos(\theta_{mid}) \quad (2.4)$$

$$y += d \times \sin(\theta_{mid}) \quad (2.5)$$

với θ_{mid} là hướng trung bình trong chu kỳ. Quãng đường mỗi xung encoder quy đổi qua chu vi bánh xe (xem Công thức 3.1 và 3.2, mục 3.2.3).

2.2.2 Bộ lọc bù (Complementary Filter)

Odometry chỉ dựa trên encoder sẽ tích lũy sai số hướng rất nhanh (trượt bánh, đường kính bánh không đều). Ngược lại, con quay hồi chuyển (gyro) của IMU đo vận tốc góc rất tốt trong ngắn hạn nhưng bị trôi (drift) khi tích phân lâu dài. Bộ lọc bù [9] kết hợp hai nguồn để bù khuyết điểm của nhau: (một sự lựa chọn phức tạp hơn là Kalman [29], [30])

$$\theta_{fused} = \theta_{enc} + \alpha \times (\theta_{imu} - \theta_{enc}) \quad (2.6)$$

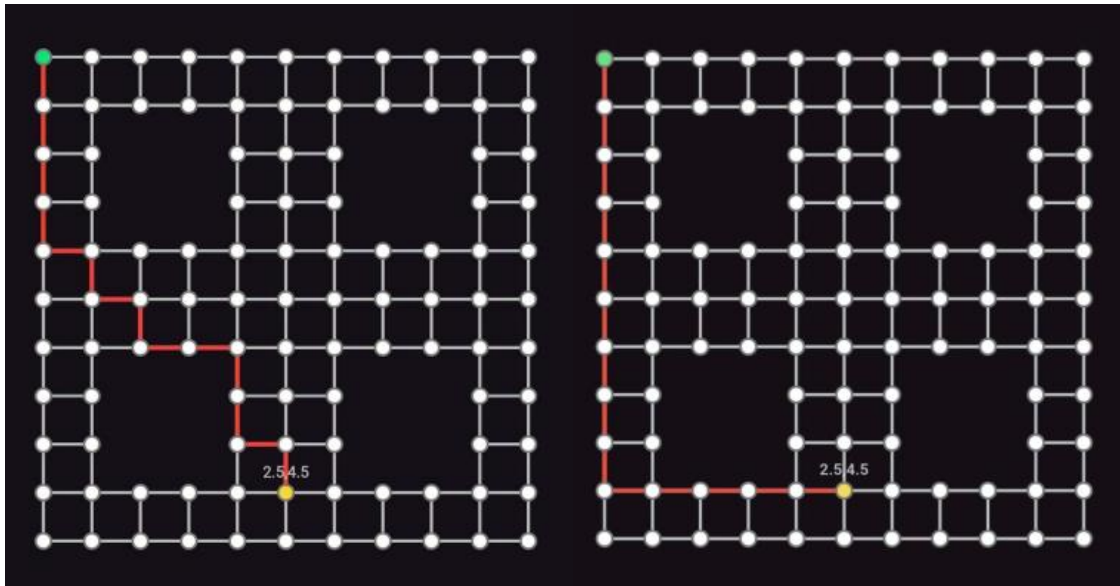
Hệ số $\alpha = 0,35$ được chọn để encoder làm chủ hướng (65%) còn IMU hiệu chỉnh (35%). Đây là lựa chọn thực nghiệm: encoder cho hướng ổn định khi đi thẳng, IMU kéo lại sai lệch khi bánh trượt. Trường hợp đặc biệt: khi xoay tại chỗ (hai bánh quay ngược chiều, bánh trượt ngang, encoder không đáng tin) thì dùng IMU 100%; khi IMU mất kết nối thì dùng encoder 100% làm phương án dự phòng.

2.2.3 Thuật toán tìm đường A*

A* (A-star) [11] , [31] là thuật toán tìm đường đi ngắn nhất trên đồ thị có trọng số, kết hợp chi phí thực tế đã đi $g(n)$ và một hàm ước lượng (heuristic) $h(n)$ tới đích: [6]

$$f(n) = g(n) + h(n) \quad (2.7)$$

Trong đề tài, đồ thị là lưới waypoint, trọng số mỗi cạnh là khoảng cách Euclid giữa hai node; heuristic $h(n)$ cũng là khoảng cách Euclid từ node tới đích - đây là heuristic chấp nhận được (admissible) vì khoảng cách đường chim bay không bao giờ lớn hơn quãng đường thực, đảm bảo A* tìm ra đường tối ưu. Đề tài còn bổ sung một chi phí phạt rẽ (turn penalty): khi hai đường có cùng độ dài, A* ưu tiên đường ít phải rẽ hơn để xe chạy mượt và giảm sai số tích lũy do mỗi lần xoay. Với môi trường thay đổi rộng, các phương án tái lập lại kế hoạch như D* Lite [32] cho phép cập nhập đường đi tăng tiến mà không phải tính lại từ đầu.



Hình 2.4.1 Hình ảnh của tính toán quãng đường đi A* mà không có sự tối ưu hóa góc rẽ.

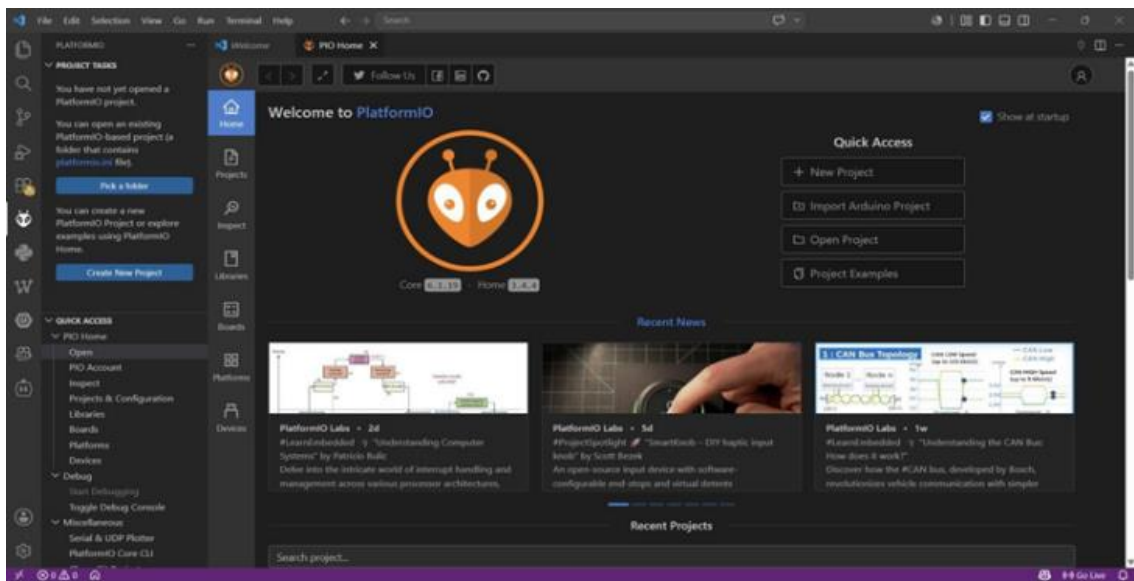
Hình 2.4.2 Hình ảnh của tính toán quãng đường đi A* có sự tối ưu hóa góc rẽ.

Hình 2.4: So sánh kết quả lập kế hoạch đường đi A trước và sau khi tối ưu góc rẽ

2.3 CÁC PHẦN MỀM XÂY DỰNG VÀ HỖ TRỢ

2.3.1 PlatformIO trên Visual Studio Code

Firmware ESP32 được phát triển bằng PlatformIO [33] - một hệ sinh thái phát triển nhưng chạy trên VS Code, hỗ trợ quản lý thư viện, biên dịch, nạp chương trình và giám sát Serial. So với Arduino IDE truyền thống, PlatformIO mạnh hơn ở quản lý dự án nhiều file (module hóa), khai báo thư viện qua platformio.ini , và hỗ trợ nhiều môi trường build (đề tài có 2 môi trường: esp32dev cho firmware chính, l298n_test để test riêng mạch lái động cơ, servo_test để test servo và HCSR-04). Ngôn ngữ sử dụng là C++ trên framework Arduino-ESP32. [12]



Hình 2.5: Giao diện PlatformIO trên VS Code

2.3.2 Flutter và ngôn ngữ Dart

Ứng dụng di động điều khiển được xây dựng bằng Flutter (ngôn ngữ Dart) của Google, framework đa nền tảng, từ một mã nguồn có thể build cho Android, iOS, Windows... Flutter cho phép vẽ giao diện bản đồ tùy biến (CustomPainter), xử lý chạm, quản lý trạng thái và giao tiếp UDP. Đề tài tận dụng Flutter để dựng 4 màn hình: Bản đồ, Kiểm tra kết nối, Điều khiển tay, và Chế độ tự hành.

2.3.3 Kicad (thiết kế mạch)

Phần mềm thiết kế mạch (Kicad) được dùng để vẽ sơ đồ nguyên lý kết nối ESP32 với các module và làm cơ sở cho việc đi dây/làm mạch. Do hệ thống chủ yếu lắp ghép từ các module có sẵn (L298N, MPU-6050, HC-SR04...), sơ đồ tập trung thể hiện kết nối chân giữa các khối.

2.4 CÁC THƯ VIỆN SỬ DỤNG

Firmware (C++ / Arduino-ESP32): các thư viện được sử dụng được liệt kê dưới bảng sau

Bảng 2.1: Các thư viện dùng trong firmware

Thư viện	Vai trò trong đề tài
WiFi.h (built-in)	Tạo WiFi Access Point (SoftAP) cho ESP32
WiFiUdp.h (built-in)	Mở socket UDP gửi telemetry / nhận lệnh
Wire.h (built-in)	Giao tiếp I2C mức thấp với MPU-6050 (đọc/ghi thanh ghi trực tiếp, không dùng thư viện MPU bên ngoài)
Preferences.h (built-in)	Lưu dữ liệu hiệu chuẩn (motor trim, encoder scale) vào NVS flash [34]
Hàm ledc* , attachInterrupt , pulseIn (core)	PWM phân cứng, ngắt encoder, đo xung siêu âm

Ứng dụng (Dart / Flutter): các thư viện được sử dụng được liệt kê dưới bảng sau

Bảng 2.2: Các thư viện dùng trong App Flutter

Gói (package)	Phiên bản	Vai trò
Udp	^2.0.0	Socket UDP gửi lệnh / nhận telemetry
path_provider	^2.1.0	Lấy đường dẫn file để ghi log CSV

Đáng chú ý: việc giao tiếp MPU-6050 được viết thủ công ở mức thanh ghi (đọc trực tiếp 14 byte từ thanh ghi 0x3B, tự đánh thức chip, đặt dải đo $\pm 250^\circ/\text{s}$, lọc DLPF $\sim 44\text{ Hz}$) thay vì dùng thư viện DMP, giúp hiểu sâu nguyên lý và chủ động xử lý lỗi đọc/mất kết nối.

CHƯƠNG 3: THIẾT KẾ VÀ THI CÔNG

3.1 YÊU CẦU VÀ SƠ ĐỒ KHỐI VÀ HỆ THỐNG

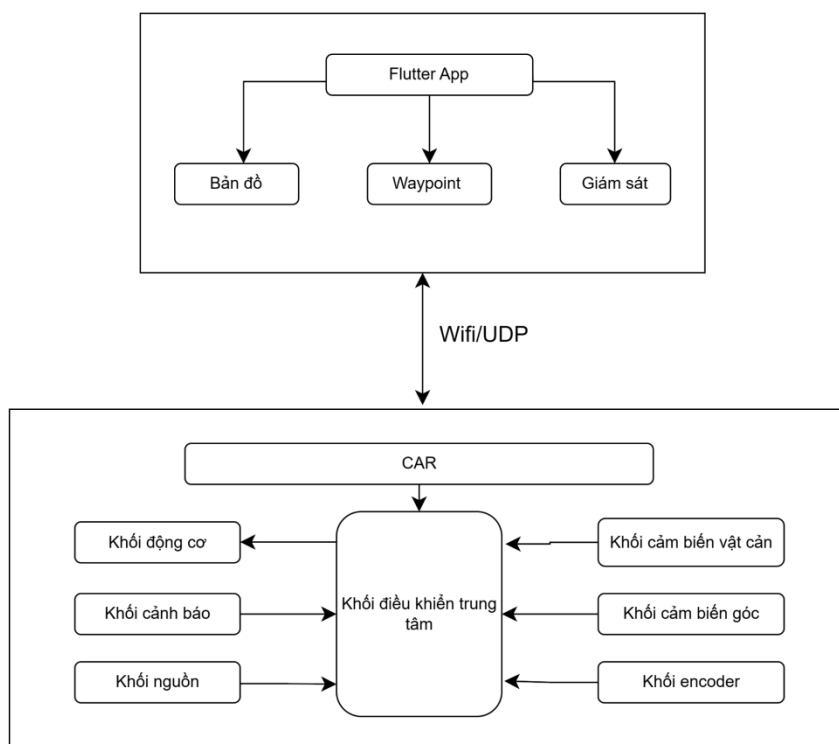
3.1.1 Yêu cầu của hệ thống

Hệ thống cung cấp các chức năng sau:

- Người dùng chạm chọn điểm đích trên bản đồ trong ứng dụng; ứng dụng tự tính đường đi ngắn nhất (A^*) và gửi lộ trình xuống xe.
- Xe tự bám theo lộ trình: tự xoay đúng hướng tại mỗi node và đi thẳng đúng quãng đường giữa các node, dựa trên odometry (encoder + IMU).
- Hiển thị vị trí xe theo thời gian thực trên bản đồ (telemetry 10 Hz), kèm trạng thái hoạt động và vệt đường đã đi.
- Điều khiển tay (tiến/lùi/rẽ/đi chéo) và chế độ test từng bước phục vụ kiểm tra.
- Tự hiệu chuẩn sai số cân bằng động cơ và tỉ lệ encoder, lưu lại qua các lần khởi động.
- Chế độ tự hành né vật cản: xe tự chạy thẳng và dùng siêu âm HC-SR04 trên servo quét để phát hiện, đo và tự né vật cản (rẽ 90° về bên thoáng hơn, quay đầu 180° hoặc dừng khi bị kẹt) - bật/tắt bằng nút trên ứng dụng. Đây là chế độ hoạt động độc lập với chế độ bám lộ trình A^* .
- Hoạt động được không cần phân cứng nhờ chế độ giả lập (simulator) trong ứng dụng.

3.1.2 Sơ đồ khối và chức năng mỗi khối

Sơ đồ khối tổng thể của hệ thống được trình bày ở Hình 3.1. Hệ thống gồm hai phần lớn: phần firmware trên xe (ESP32) và phần ứng dụng trên điện thoại, liên lạc qua WiFi/UDP



Hình 3.1: Sơ đồ khối hệ thống

3.1.3 Chức năng của từng khối

- Khối xử lý trung tâm (ESP32): bộ xử lý của xe. Nhận lệnh từ app, đọc toàn bộ cảm biến, chạy odometry và máy trạng thái điều hướng, xuất tín hiệu điều khiển động cơ, gửi telemetry về app.
- Khối động cơ (L298N + 4 DC motor): nhận tín hiệu PWM + chiều quay từ ESP32 để dẫn động bốn bánh chủ động, gộp thành 2 bánh trái phải.
- Khối encoder (2 encoder quang): đo số vòng/xung quay mỗi bên cung cấp quãng đường cho odometry.
- Khối cảm biến góc IMU (MPU-6050): đo vận tốc góc để tính hướng (yaw) chính xác, hợp nhất với encoder qua bộ lọc bù.
- Khối cảm biến khoảng cách (HC-SR04): đo khoảng cách vật cản phía trước; trong chế độ tự hành, cảm biến được gắn trên servo để quét trái/phải tìm hướng thoát. [5]
- Khối servo quét cảm biến: xoay cảm biến HC-SR04 sang trái/giữa/phải, giúp xe "nhìn" được khoảng cách hai bên mà không cần xoay cả thân xe.

- Khối cảnh báo (buzzer): phát tiếng "bíp" báo các sự kiện (tới node, tới đích, lỗi).
- Khối nguồn (pin): cấp nguồn cho động cơ và mạch điều khiển.
- Ứng dụng di động: giao diện người dùng - chọn đích, tính đường A*, gửi lộ trình, hiển thị vị trí xe và trạng thái theo thời gian thực; điều khiển tay, hiệu chuẩn và bật/tắt chế độ tự hành.

3.1.4 Các hoạt động của hệ thống

Khi khởi động, ESP32 phát WiFi AP (SSID RobotCar , IP 192.168.4.1), hiệu chuẩn IMU (đo bias gyro khi xe đứng yên) và đặt gốc tọa độ tại vị trí hiện tại. Người dùng mở ứng dụng, kết nối điện thoại vào mạng RobotCar, rồi chạm chọn node đích trên bản đồ. Ứng dụng tìm node gần điểm chạm nhất, chạy A* từ vị trí xe tới đích, dựng danh sách waypoint (tọa độ mét) và gửi xuống ESP32 qua UDP (gửi lặp để chống mất gói).

ESP32 nhận lộ trình và đưa vào Node Navigator - một máy trạng thái: tại mỗi node, xe tính hướng cần đi và xoay đúng hướng (cơ chế 3 tầng: bỏ qua nếu lệch nhỏ, xoay nhanh nếu lệch lớn, nhích tinh chỉnh nếu lệch vừa), rồi đi thẳng đúng quãng đường (đếm xung encoder) trong khi giữ hướng bằng IMU. Mỗi 20 ms, odometry cập nhật vị trí; mỗi 100 ms, ESP32 gửi telemetry (x, y, hướng, trạng thái) về app để vẽ vị trí xe. Khi tới node cuối, xe dừng và phát 3 tiếng bíp báo "đã tới đích" (arrived).

Ngoài luồng bám lộ trình trên, hệ thống có thêm chế độ tự hành né vật cản do người dùng bật từ ứng dụng. Khi bật, ESP32 trao quyền điều khiển cho khối Autonomous Explorer: xe tự chạy thẳng và đo khoảng cách phía trước; gặp vật cản gần thì dừng, dùng servo quét đo hai bên rồi tự chọn hướng đi mới (chi tiết thuật toán ở mục 3.6.6). Hai chế độ (bám lộ trình A* và tự hành né vật cản) loại trừ lẫn nhau - khi nhận lệnh chế độ này thì chế độ kia tự dừng.

3.2 THIẾT KẾ PHẦN CỨNG

Bảng 3.1 Bảng phân công chân GPIO cho các thiết bị ngoại vi

Khối	Tín hiệu	Chân ESP32
------	----------	------------

Động cơ trái	IN1 / IN2 / ENA (PWM)	GPIO12 / GPIO13 / GPIO14
Động cơ phải	IN3 / IN4 / ENB (PWM)	GPIO26 / GPIO27 / GPIO25
Encoder	Trái / Phải (D0)	GPIO34 / GPIO32
IMU MPU-6050	SDA / SCL (I2C)	GPIO21 / GPIO22
Siêu âm HC-SR04	Trig / Echo	GPIO5 / GPIO18
Servo quét cảm biến	Tín hiệu PWM (LEDC kênh 2)	GPIO19
Cảnh báo	Buzzer	GPIO4

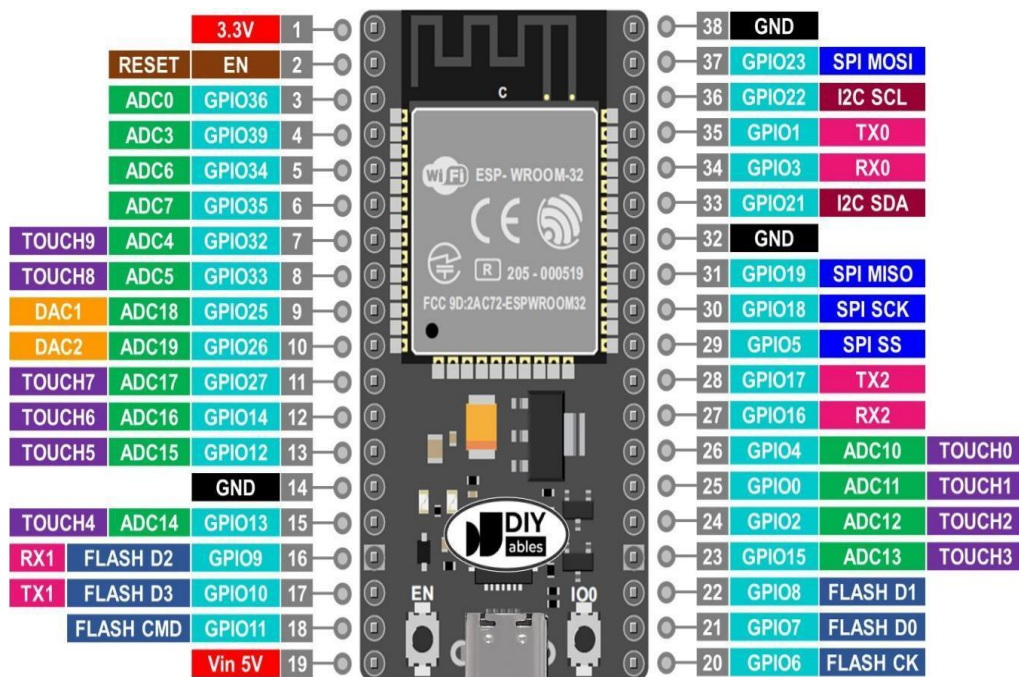
3.2.1 Khối xử lý trung tâm (ESP32)

Chức năng: trung tâm điều khiển, chạy toàn bộ firmware ở chu kỳ vòng lặp 50 Hz (20 ms).

Lựa chọn linh kiện: đề tài sử dụng board ESP32 DevKit (ESP32-WROOM-32, 38 chân) [35] vì các lý do:

- Hiệu năng cao: lõi kép Xtensa 240 MHz, đủ sức chạy đồng thời odometry, máy trạng thái điều hướng và UDP.
- Tích hợp WiFi: phát được Access Point để app kết nối trực tiếp, không cần module mạng ngoài - đây là yếu tố then chốt giúp loại bỏ router.
- Nhiều GPIO và ngoại vi: đủ chân cho 1 động cơ (6 chân), 2 encoder, I2C, siêu âm, buzzer; có bộ LEDC tạo PWM phần cứng và nhiều chân hỗ trợ ngắt.
- Hỗ trợ NVS flash: lưu dữ liệu hiệu chuẩn không mất khi tắt nguồn.

- Phổ biến, chi phí thấp, cộng đồng lớn, dễ lập trình bằng C++ trên PlatformIO.



Hình 3.2 Sơ đồ chân của ESP32-WROOM-32

3.2.2 Khối động cơ (mạch lái L298N + 4 động cơ DC)

Chức năng: chuyển tín hiệu điều khiển logic của ESP32 thành dòng đủ lớn để quay bốn động cơ DC.

Lựa chọn linh kiện: dùng module L298N (mạch cầu H đôi) [36]. L298N có hai kênh cầu H điều khiển độc lập hai bên; mỗi bên đầu song song hai động cơ lên cùng một kênh, dùng 2 chân chiều quay (IN) và 1 chân Enable nhận PWM. So với DRV8833/TB6612, L298N có sẵn, rẻ, chịu dòng tốt, tản nhiệt tích hợp, phù hợp động cơ DC giảm tốc cỡ nhỏ.

Bảng 3.2 Thông số kỹ thuật của module L298N

Thuộc tính	Giá trị
Điện áp động lực	5 – 35 V
Dòng đỉnh mỗi kênh	2 A
Số kênh (động cơ)	4 (cầu H đôi)
Tín hiệu điều khiển	Logic 3.3 V / 5 V tương thích

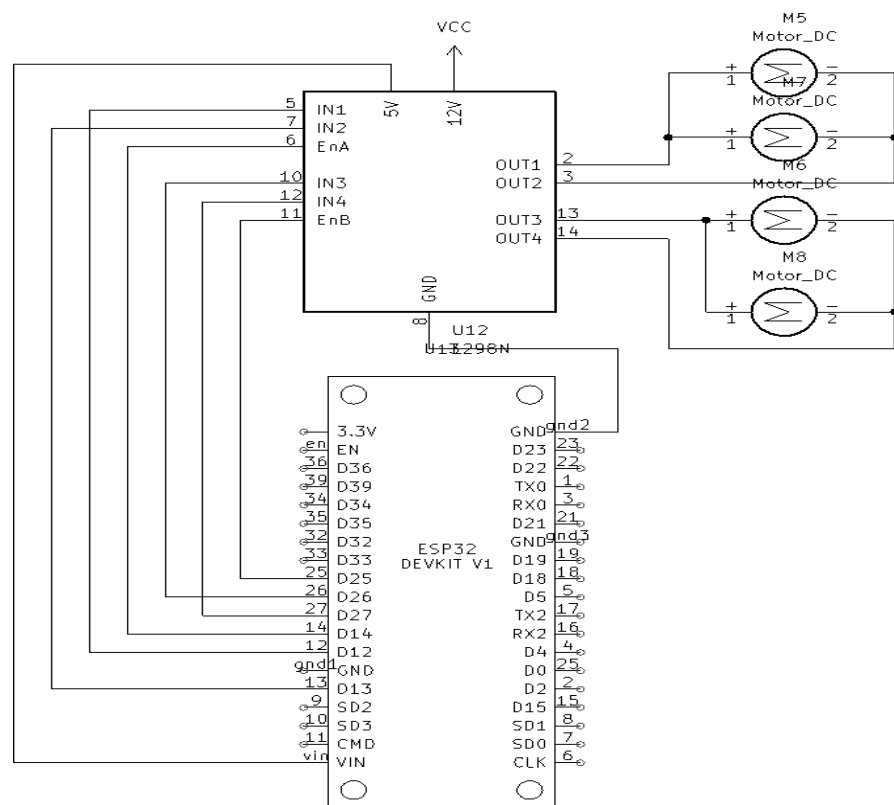
Có ổn áp 5V on-board	Có (cấp ngược lại cho ESP32)
----------------------	------------------------------

Nguyên lý điều khiển: mỗi bên có một giá trị tốc độ trong khoảng -255 đến 255.

Dấu quyết định chiều (đặt IN1/IN2 hoặc IN3/IN4), độ lớn là duty PWM cấp vào chân Enable qua kênh LEDC (5 kHz, 8-bit). Firmware còn cung cấp hàm tăng/giảm tốc từ từ (slew rate) - mỗi vòng lặp chỉ thay đổi PWM tối đa 18 đơn vị - để tránh giật khi khởi động và giảm trượt bánh.

Mô tả kết nối:

- IN1 = GPIO12, IN2 = GPIO13, ENA = GPIO14 (nhóm động cơ bên trái).
- IN3 = GPIO26, IN4 = GPIO27, ENB = GPIO25 (nhóm động cơ bên phải).
- Chân động lực 12V/VS của L298N lấy từ pin; 5V out của L298N có thể cấp ngược cho ESP32 (không khuyến khích). GND của L298N, ESP32 và pin phải nối chung (common ground)



Hình 3.3 Sơ đồ kết nối ESP32 - L298N – bốn động cơ DC (nhóm trái/phải)

3.2.3 Khối encoder

Chức năng: đo quãng đường mỗi bánh, là nguồn dữ liệu chính cho odometry. [3]

Lựa chọn linh kiện: dùng encoder quang LM393 với đĩa mã hóa 20 khe (20 xung/vòng), gắn trên trục mỗi bánh. Đây là loại encoder số một kênh (chỉ có ngõ D0), rẻ, dễ lắp; chiều quay được suy ra từ lệnh động cơ nên không cần kênh thứ hai. [11]

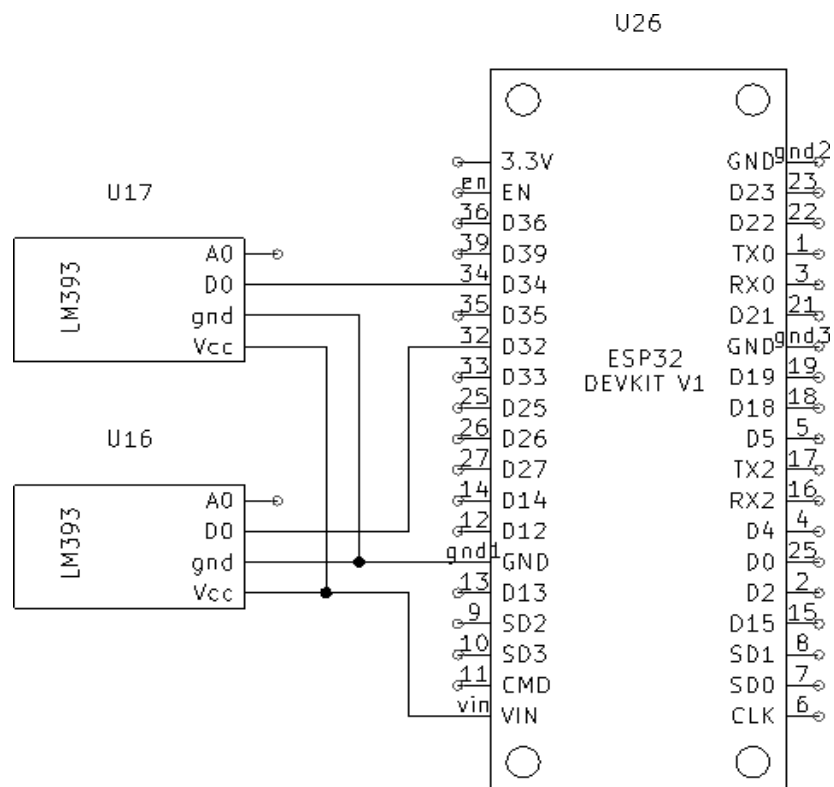
Tính toán quãng đường: với đường kính bánh 6,5 cm:

$$cm \text{ mỗi xung} = \frac{Chu \text{ vi}}{Xung \text{ mỗi vòng}} = \frac{20.42}{20} = 1.021 \text{ cm} \quad (3.1)$$

$$\text{Với: } Chu \text{ vi} = \pi \times 6.5 = 20.42 \text{ cm} \quad (3.2)$$

Khoảng cách giữa hai node là 0,5 m, tương đương lý thuyết $\frac{50}{1.021} \approx 49$ xung, tham số dựa trên đo đạc thực tế.

Xử lý nhiễu: ISR đếm xung trên cạnh lên, áp dụng chống dội 6000 μs để loại nhiễu EMI từ động cơ. Việc đọc bộ đếm được bảo vệ bằng tắt/bật ngắt tạm thời (atomic) để tránh tranh chấp với ISR.



Hình 3.4: Sơ đồ kết nối encoder LM393 với ESP32

3.2.4 Khối cảm biến góc IMU (MPU6050)

Chức năng: đo hướng (yaw) của xe để hiệu chỉnh odometry và giữ hướng khi đi thẳng / xoay đúng góc.

Lựa chọn linh kiện: dùng MPU-6050 [37] (6 trục: 3 gia tốc + 3 con quay hồi chuyển), giao tiếp I2C, giá rẻ, rất phổ biến. Đề tài chỉ dùng trục gyro Z để tính yaw.

Nguyên lý:

- Khởi động: đánh thức chip, đặt tần số lấy mẫu 100 Hz, bộ lọc DLPF ~44 Hz, dải đo con quay $\pm 250^\circ/\text{s}$ (độ nhạy 131 LSB/($^\circ/\text{s}$)).
- Hiệu chuẩn bias: khi xe đứng yên, đọc ~1000 mẫu gyro Z để tính giá trị lệch 0 (bias), loại bỏ trôi tĩnh.
- Tính yaw: mỗi chu kỳ, đọc giá trị gyro thô, trừ bias, đổi sang rad/s rồi tích phân theo thời gian.

Công thức đổi gyro thô $\rightarrow$ vận tốc góc:

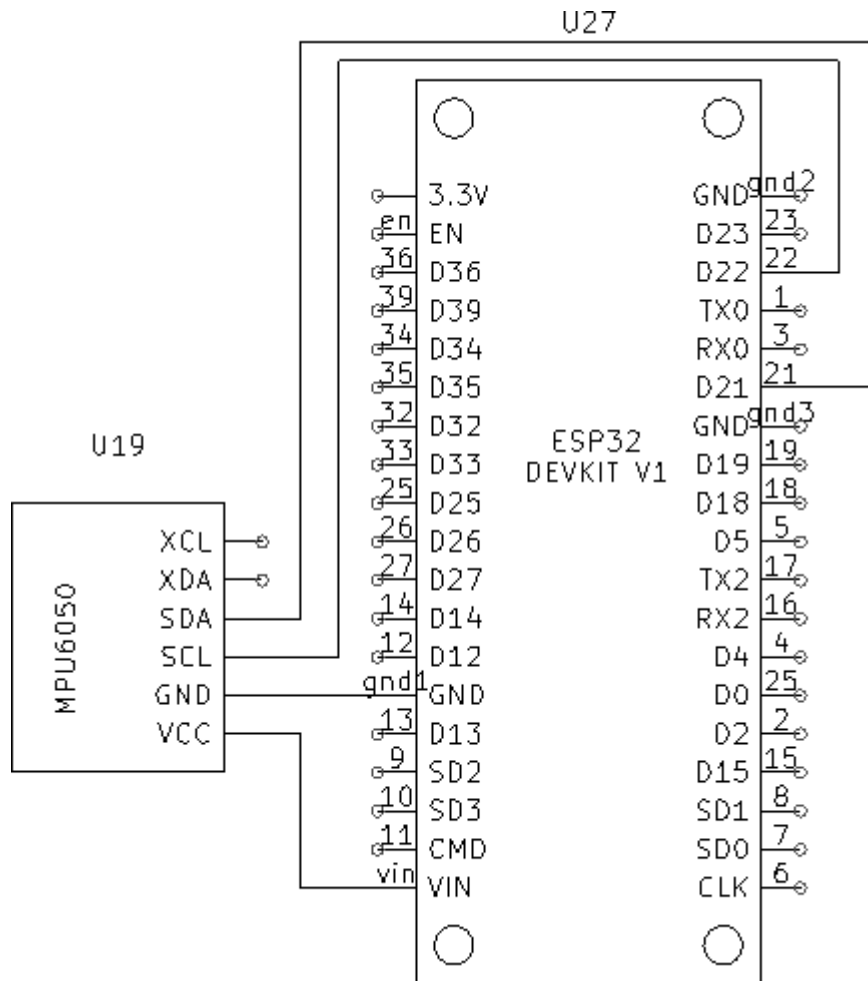
$$\omega_z = (g_{z_{raw}} - bias_z) \times \frac{1}{(131 \times \frac{180}{\pi})} \text{ (rad/s)} \quad (3.3)$$

Công thức tích phân yaw:

$$\theta_{yaw} += \omega_z \times \Delta t \quad (3.4)$$

Để chống trôi khi đứng yên, áp dụng deadband 0,01 rad/s (bỏ qua vận tốc góc rất nhỏ). Firmware còn theo dõi sức khỏe cảm biến: nếu đọc lỗi liên tiếp 5 lần thì đánh dấu "mất kết nối" và odometry tự chuyển sang dùng encoder 100%.

Mô tả kết nối: VCC $\rightarrow$ 3.3V/5V; GND $\rightarrow$ GND chung; SDA $\rightarrow$ GPIO21; SCL $\rightarrow$ GPIO22.



Hình 3.5 Sơ đồ kết nối MPU-6050 qua I2C

3.2.5 Khối cảm biến khoảng cách (HC-SR04)

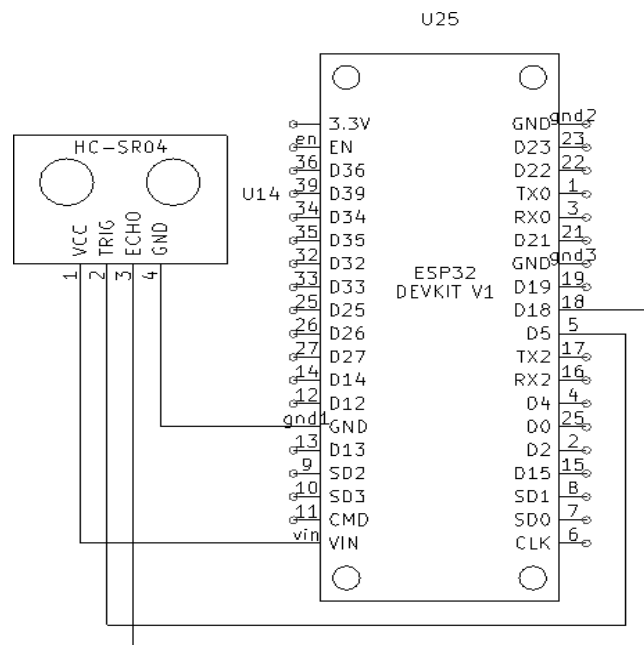
Chức năng: đo khoảng cách vật cản phía trước để phục vụ chế độ tự hành né vật cản - phát hiện chướng ngại, dừng kịp thời và đo khoảng cách hai bên (qua servo quét) để chọn hướng đi.

Lựa chọn linh kiện: dùng HC-SR04 [38] (siêu âm), giao tiếp số đơn giản (Trig/Echo), tầm đo 2–400cm, rẻ, dễ tích hợp, không nhạy với ánh sáng môi trường (khác cảm biến hồng ngoại). So với cảm biến ToF laser (VL53L0X) đắt hơn, HC-SR04 đủ dùng cho mục tiêu phát hiện vật cản cơ bản.

Nguyên lý: phát xung trigger 10 μ s, đo độ rộng xung Echo bằng pulseIn (timeout 30 ms $\approx$ ngoài tầm), quy đổi sang cm theo Công thức 2.1 (vận tốc âm $\sim 0,0343$ cm/ μ s, chia 2 vì sóng đi và về). Firmware cung cấp hai cách đọc tùy ngữ cảnh:

- Đọc kỹ (readDistanceCm) - lấy trung vị của 5 mẫu (median filter, có nghỉ giữa các mẫu) để khử nhiễu; dùng khi xe đứng yên quét hai bên, nơi cần độ chính xác cao.
- Đọc nhanh (readQuickCm) - chỉ một lần đo, không nghỉ, để không làm nghẽn vòng lặp 50 Hz khi xe đang chạy thẳng và cần đo liên tục. Khi phát hiện vật cản, firmware đọc thêm một lần xác nhận để loại nhiễu một lần trước khi quyết định dừng.

Ngưỡng quyết định trong chế độ tự hành: vật cản phía trước < 30 cm thì dừng và quét; cản ≥ 25 cm ở cả hai bên mới đủ chỗ quay đầu 180° (chi tiết ở mục 3.6.6). Ngoài ra mã nguồn vẫn giữ tùy chọn dừng khẩn cấp độc lập (USE_ULTRASONIC, ngưỡng 10 cm / cảnh báo 20 cm) dành cho chế độ bám lộ trình, mặc định tắt để không làm chậm vòng lặp khi không cần.



Hình 3.6 Sơ đồ kết nối HC-SR04 với ESP32

3.2.6 Khối servo quét cảm biến

Chức năng: xoay cảm biến HC-SR04 sang trái / giữa / phải để xe xác định được khoảng cách ba hướng mà không cần xoay cả thân xe - giúp khâu quyết định né vật cản nhanh và ít hao pin hơn. [10]

Lựa chọn linh kiện: dùng servo SG90 (xoay 180°), điều khiển bằng một dây tín hiệu PWM 50 Hz, giá rẻ và rất phổ biến. Thông số chính: điện áp 4,8–6 V, mô-men ~1,2–2,4 kg·cm, đủ để xoay nhẹ cụm cảm biến HC-SR04; trọng lượng nhỏ (~9 g) phù hợp gắn ở mũi xe.

Bảng 3.3: Thông số servo SG90 và tham số chế độ tự hành

Thông số	Giá trị
Loại servo / góc quay	SG90 / 0-180°
Điện áp hoạt động	4,8 - 6 V
Mô-men	~1,2 -2,4 kg·cm
Tín hiệu điều khiển	PWM 50 Hz (xung 0,5–2,5 ms), LEDC kênh 2, 16-bit
Góc giữa / trái / phải	90° / 180° / 0°
Thời gian chờ servo ổn định	~350 ms trước mỗi lần đo
Ngưỡng dừng phía trước	< 30 cm
Khoảng trống tối thiểu để quay đầu	≥ 25 cm hai bên
Tốc độ chạy thẳng tự hành	PWM 160
Góc né khi rẽ	90° (trái/phải) hoặc 180° (quay đầu)

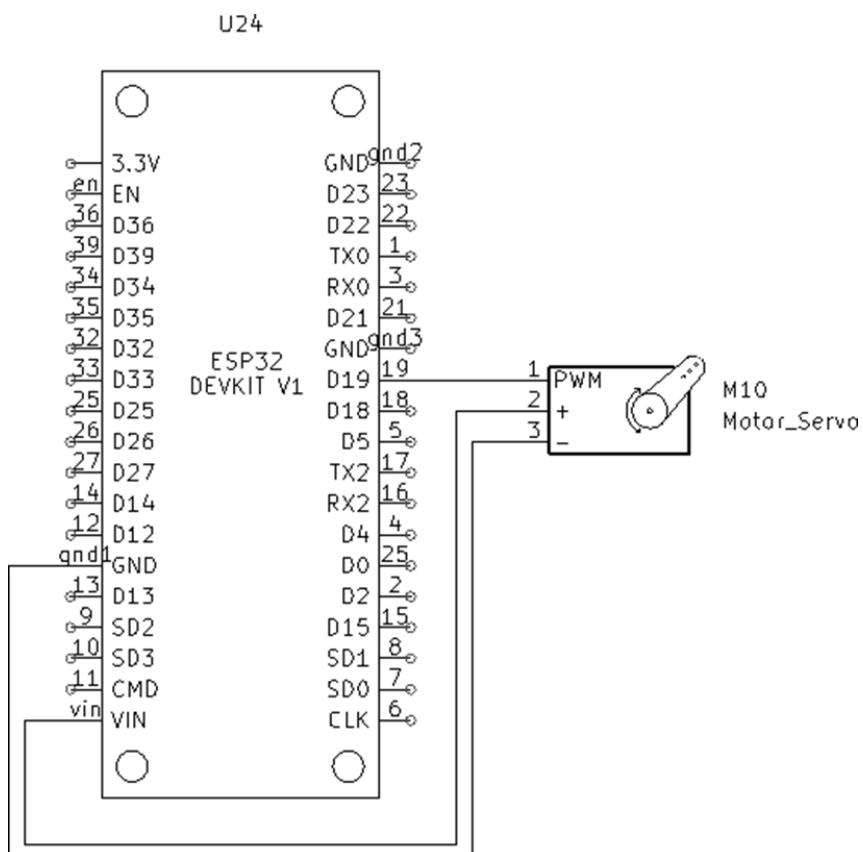
Nguyên lý điều khiển: thay vì dùng thư viện servo ngoài, đề tài điều khiển trực tiếp bằng LEDC thông qua kênh 2 riêng (50 Hz, độ phân giải 16-bit) để đồng bộ phong cách với khối động cơ và tránh xung đột bộ định thời. Góc mong muốn (0-180°) được quy đổi sang độ rộng xung rồi sang duty LEDC:

Công thức góc servo-> xung-> duty:

$$pulse(\mu s) = 500 + \frac{(2000 \times \text{góc})}{180} \quad (3.5)$$

$$duty = pulse \times \frac{(2^{16}-1)}{20000} \quad (3.6)$$

Ba vị trí dừng trong chế độ tự hành: giữa = 90° (nhìn thẳng khi chạy), trái = 180°, phải = 0°. Sau mỗi lần ra lệnh xoay, firmware chờ ~350 ms cho servo ổn định rồi mới đo siêu âm để tránh đọc sai lúc cánh tay còn đang quay. (Các góc trái/phải có thể tinh chỉnh theo cách lắp cơ khí thực tế.)



Hình 3.7: Sơ đồ kết nối servo quét cảm biến với ESP32

3.2.7 Khỏi cảnh báo (buzzer)

Chức năng: phản hồi âm thanh cho người dùng. Buzzer (chân GPIO4) phát các chuỗi bíp ngắn báo sự kiện: 1 bíp khi qua một node, 3 bíp khi tới đích (arrived).

Cơ chế bíp được lập lịch không chặn (non-blocking): bật 75 ms, nghỉ 95 ms giữa các tiếng, không làm gián đoạn vòng lặp điều khiển.

3.2.8 Khỏi nguồn

Hệ thống dùng nguồn pin cấp cho cả động lực và điều khiển. Bảng 3.3 liệt kê mức điện áp các thiết bị.

Bảng 3.4: Các giá trị nguồn của thiết bị

Thiết bị	Điện áp hoạt động
ESP32	3.3 V (logic) / 5 V (cấp qua VIN)
L298N (động lực)	5 – 35 V (đề tài dùng ~7.4 V pin)
Động cơ DC giảm tốc	3.3 – 6 V
MPU-6050	3.3 – 5 V
HC-SR04	5 V
Servo SG90/MG90S	5 V (nên cấp riêng thông qua giảm áp 7,4V ->5V của L298N)
Encoder LM393	3.3 – 5 V
Buzzer	3.3 – 5 V

Phương án cấp nguồn: pin (ví dụ 2 cell Li-ion 18650, ~7.4 V) cấp vào chân động lực L298N; ngõ ổn áp 5V của L298N cấp ngược cho ESP32 (qua VIN). Toàn bộ GND nối chung. ESP32 tự có các bộ ổn áp tạo mức 3.3 V cho logic và cảm biến. Lưu ý phần mềm có tắt cảnh báo sụt áp (brown-out) của ESP32 để tránh reset khi động cơ khởi động kéo dòng lớn.

3.3 THIẾT KẾ PHẦN MỀM ỨNG DỤNG DI ĐỘNG (FLUTTER)

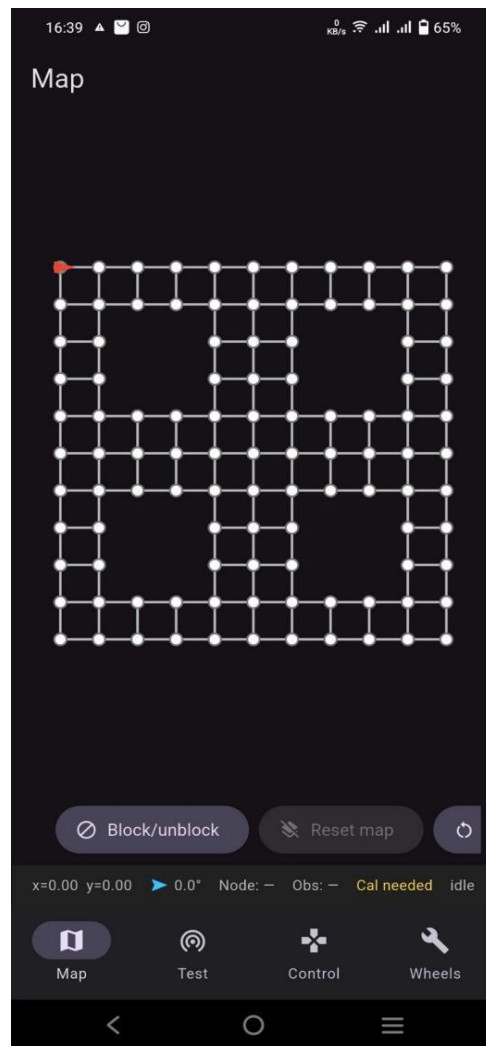
Ứng dụng được tổ chức theo mô hình từng lớp rõ ràng (models - planning - services - screens), giúp dễ kiểm thử và bảo trì.

Các lớp chính:

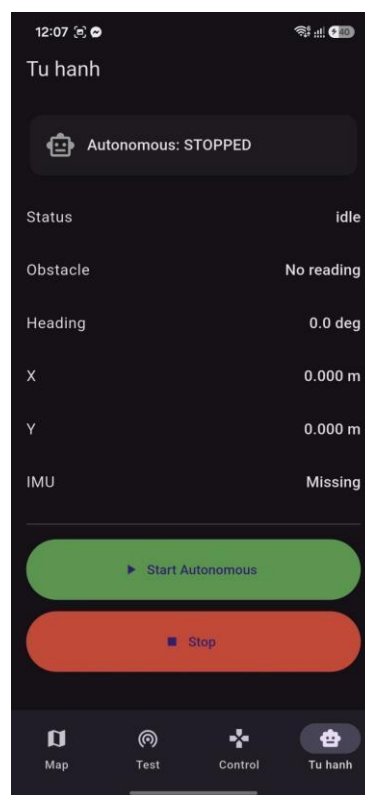
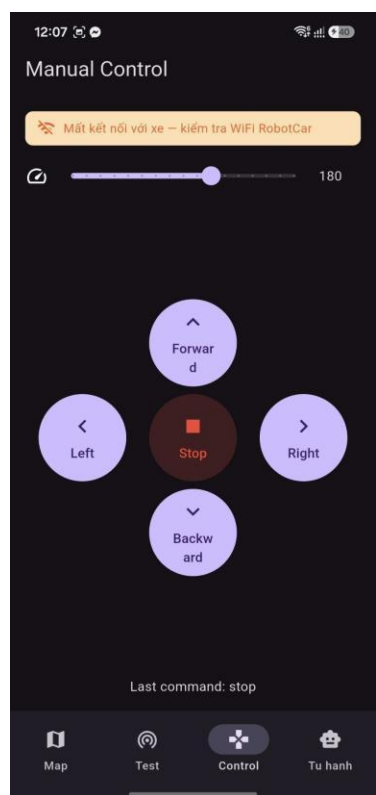
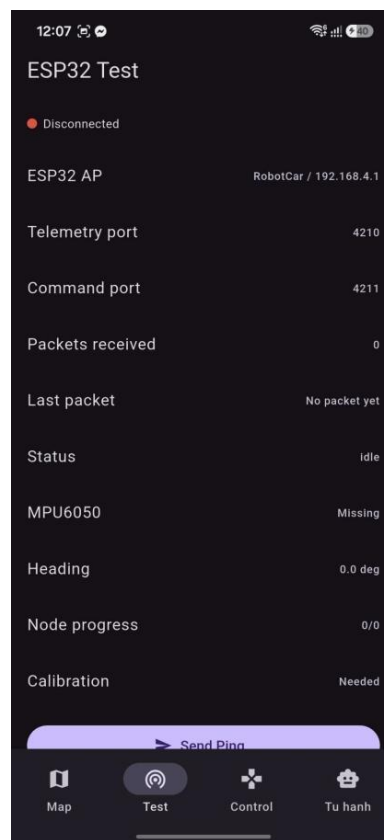
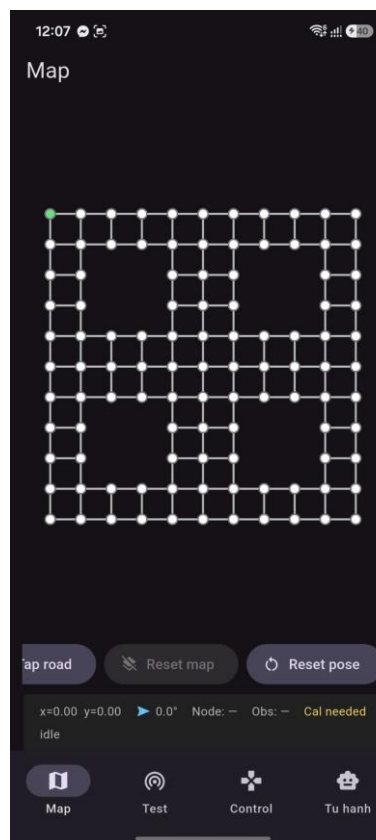
- Models: RobotState (pose x, y, θ ; khoảng cách vật cản; trạng thái - kể cả các trạng thái hành exploring/ scanning /avoiding /stuck), RoadMap (Node, Edge, các cạnh bị chặn lúc chạy).
- planning: A* (tìm đường), RoadMap Loader (đọc bản đồ JSON), WaypointBuilder (đổi danh sách thành danh sách tọa độ mét).
- services: UdpService (gửi lệnh / nhận telemetry qua UDP, có hàm sendAutonomousCommand bật/tắt chế độ tự hành) và SimulatorService (giả lập xe chạy 0,5 m/s khi không có phần cứng) - cùng hiện thực giao diện ITransportService nên có thể hoán đổi.
- screens: 4 màn hình điều hành.

Bốn màn hình ứng dụng

- Bản đồ (Map): Vẽ đồ thị waypoint, vị trí xe thời gian thực + vết đường đã đi; chạm chọn đích $\rightarrow$ A* $\rightarrow$ gửi lộ trình. Có chế độ chặn đường (Block/unblock): chạm một cạnh để chặn/mở lại, A* tính lại lộ trình tránh đoạn bị chặn (mô phỏng vật cản trên bản đồ); kèm hai nút Reset map (xóa các cạnh đã chặn) và Reset pose (đưa xe về gốc tọa độ).
- Kiểm tra & hiệu chuẩn (ESP32 Test): màn hình chẩn đoán kết nối và hiệu chuẩn. Hiển thị địa chỉ AP, cổng telemetry/command, số gói nhận được, gói cuối, trạng thái, MPU-6050, hướng, tiến trình node và tình trạng hiệu chuẩn. Các nút: Send Ping, Calibrate IMU (hiệu chuẩn cảm biến góc), Reset Motor Auto-Tune (đặt lại hệ số cân bằng động cơ) và Reset Encoder Scale (đặt lại tỉ lệ encoder).
- Điều khiển tay (Manual Control): thanh trượt chỉnh tốc độ (PWM) cùng các nút tiến / lùi / rẽ trái / rẽ phải / dừng; hiển thị lệnh vừa gửi.
- Tự hành (Autonomous): nút Start/Stop bật/tắt chế độ tự hành né vật cản, kèm bảng telemetry trực tiếp (trạng thái, khoảng cách vật cản, hướng, tọa độ X/Y, tình trạng IMU).



Hình 3.8: Bản đồ lưới waypoint



Hình 3.9: Các giao diện chính của ứng dụng điều khiển x

3.4 GIAO THỨC TRUYỀN THÔNG UDP

ESP32 phát WiFi AP: SSID RobotCar , mật khẩu 12345678 , IP cố định chiều dữ liệu dùng hai cổng UDP riêng:

Bảng 3.5: Phân bổ cổng và chức năng truyền thông

Hướng	Cổng	Tần suất	Nội dung
ESP32 → App (telemetry)	4210	10 Hz	Vị trí, hướng, trạng thái, khoảng cách vật cản
App → ESP32 (lệnh)	4211	Khi cần	Lộ trình, điều khiển tay, tự hành, hiệu chuẩn, reset

Gói telemetry (JSON, ESP32 → App):

```
{"pose":{"x":0.3500,"y":0.1200,"theta":1.5700},  
  "heading_deg":90.0,"node_index":2,"route_size":5,  
  "obstacle_dist_cm":200.0,"status":"moving",  
  "imu_connected":true,"calibrated":true}
```

(Trường obstacle_dist_cm báo khoảng cách phía trước đo được khi đang ở chế độ tự hành; ngoài chế độ này (bám lộ trình thường) nó báo giá trị tối đa 200.0 cm vì siêu âm không được đọc liên tục.

Gói lệnh lộ trình (JSON, App → ESP32):

```
{"seq":1,"waypoints":[{"x":0.5,"y":0.0}, {"x":1.0,"y":0.0}]}
```

Gói lệnh bật/tắt tự hành (JSON, App → ESP32):

```
{"seq":42,"type":"autonomous","command":"start"}
```

Trường command nhận "start" (bật chế độ tự hành né vật cản) hoặc đưa servo về giữa).

Cơ chế đảm bảo tin cậy trên nền UDP:

- Chống trùng lặp (dedup): mỗi lệnh có số thứ tự seq ; ESP32 ghi nhớ seq cuối, bỏ qua nếu trùng. App gửi lặp một lệnh nhiều lần để chống mất gói mà không gây thực thi nhiều lần.
- Chia mảnh lộ trình (route_chunk): lộ trình dài được chia thành nhiều gói nhỏ (kèm chunk_index, chunk_count, start, total) ESP32 ghép lại

đủ mới thực thi - tránh cản kích thước gói UDP.

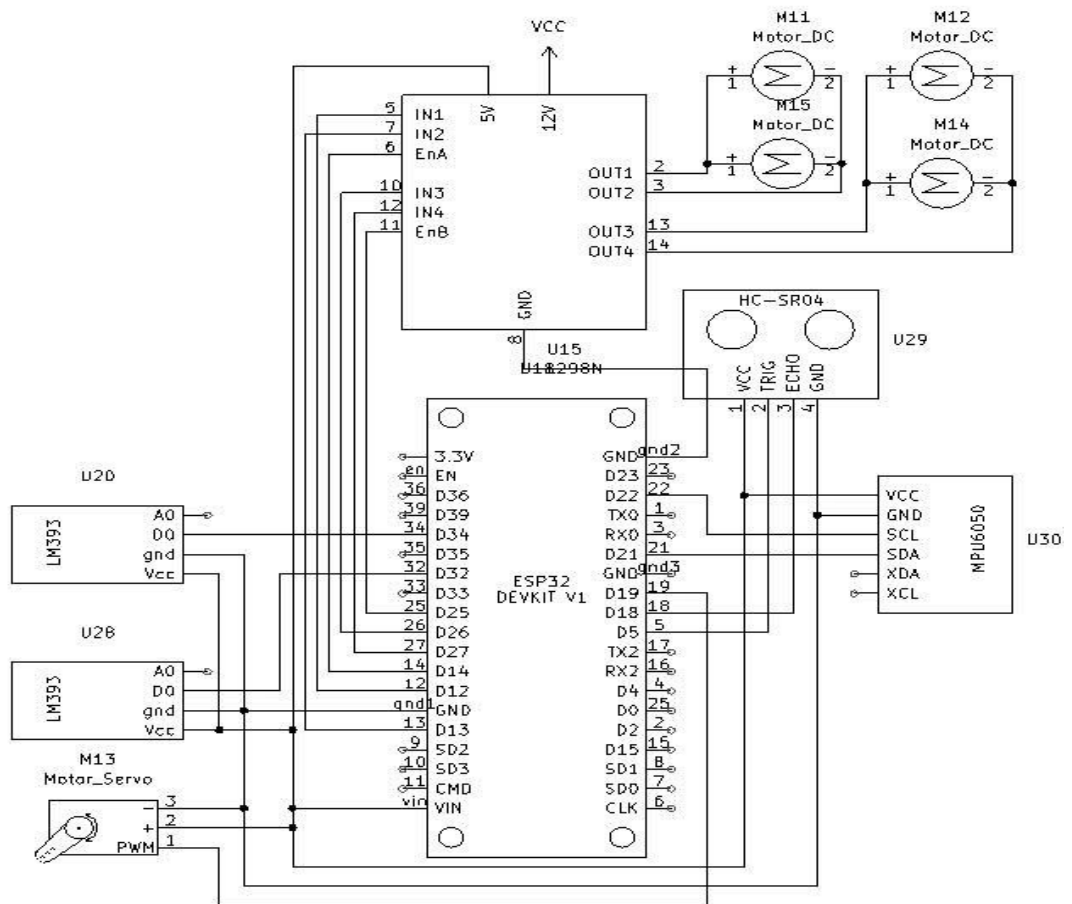
- Các loại lệnh khác: manual (điều khiển tay, kèm tốc độ và lệnh hướng), autonomous (bật/tắt tự hành né vật cản), step (test một bước), reset_pose (đặt lại gốc tọa độ), calibrate (hiệu chuẩn IMU/động cơ/encoder), ping . Khi nhận autonomous start , các chế độ khác (tay/lộ trình) tự dừng; ngược lại bất kỳ lệnh tay/lộ trình/reset nào cũng tự tắt chế độ tự hành.

Bảng 3.6: Các giá trị trạng thái (status) trong telemetry

Trạng thái Ý nghĩa	Ý nghĩa
idle	Đứng yên, sẵn sàng
moving	Đang di chuyển (xoay hoặc đi thẳng)
arrived	Đã tới đích
exploring	Tự hành: đang chạy thẳng dò đường
scanning	Tự hành: đang quét servo đo hai bên
avoiding	Tự hành: đang xoay né vật cản (90° hoặc 180°)
stuck	Tự hành: bị kẹt (cả ba phía đều hẹp), đã dừng
emergency_stop	Dừng khẩn cấp do vật cản
imu_missing	Mất kết nối IMU (đang chạy dự phòng encoder)
calibrating / calibrated	Đang / đã hiệu chuẩn
rotation_timeout	Lỗi: xoay quá lâu (motor yếu/kẹt)

3.5 SƠ ĐỒ NỐI DÂY HỆ THỐNG

Sơ đồ nguyên lý toàn hệ thống thể hiện ESP32 ở trung tâm, kết nối tới L298N (6 dây điều khiển), hai encoder (2 dây tín hiệu), MPU-6050 (2 dây I2C), HC-SR04 (2 dây Trig/Echo), servo quét cảm biến (1 dây tín hiệu GPIO19), buzzer (1 dây) và khối nguồn. Toàn bộ tuân theo phân bổ chân ở Bảng 3.1. HC-SR04 được gắn lên cánh tay servo để quay theo servo.



Hình 3.10: Sơ đồ nguyên lý toàn hệ thống

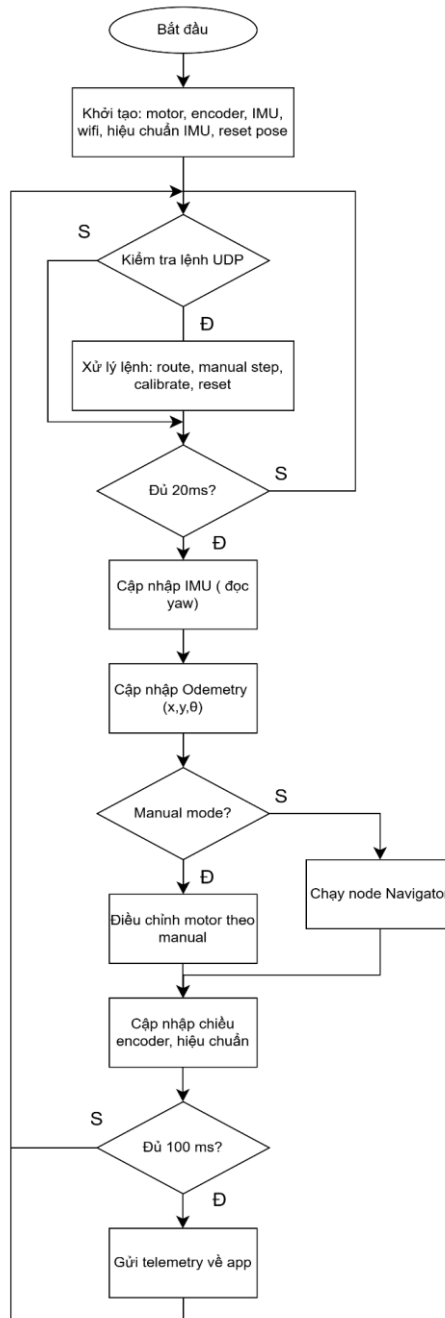
Bảng 3.7: Các tham số điều khiển chính của firmware

Tham số	Giá trị	Ý nghĩa
Chu kỳ vòng lặp	20 ms (50 Hz)	Tần suất cập nhật điều khiển/odometry
Chu kỳ telemetry	100 ms (10 Hz)	Tần suất gửi vị trí về app
Track width	10,5 cm	Khoảng cách giữa 2 bánh trái, phải (tính góc quay)
cm/xung	1,021 cm	Quãng đường mỗi xung encoder
Xung/node	52	Số xung đi hết 0,5 m
ALPHA (lọc bù)	0,35	Tỷ trọng IMU khi hợp nhất hướng
Tốc độ đi thẳng	PWM 175 (135–220)	Tốc độ cơ sở, giảm dần khi gần đích
Ngưỡng bỏ qua xoay	10°	Lệch nhỏ hơn → đi thẳng luôn
Ngưỡng xoay nhanh	25°	Lớn hơn → xoay nhanh trước
Dung sai xoay xong	4°	Sai số hướng chấp nhận được

3.6 LƯU ĐỒ GIẢI THUẬT

3.6.1 Lưu đồ giải thuật chính (firmware)

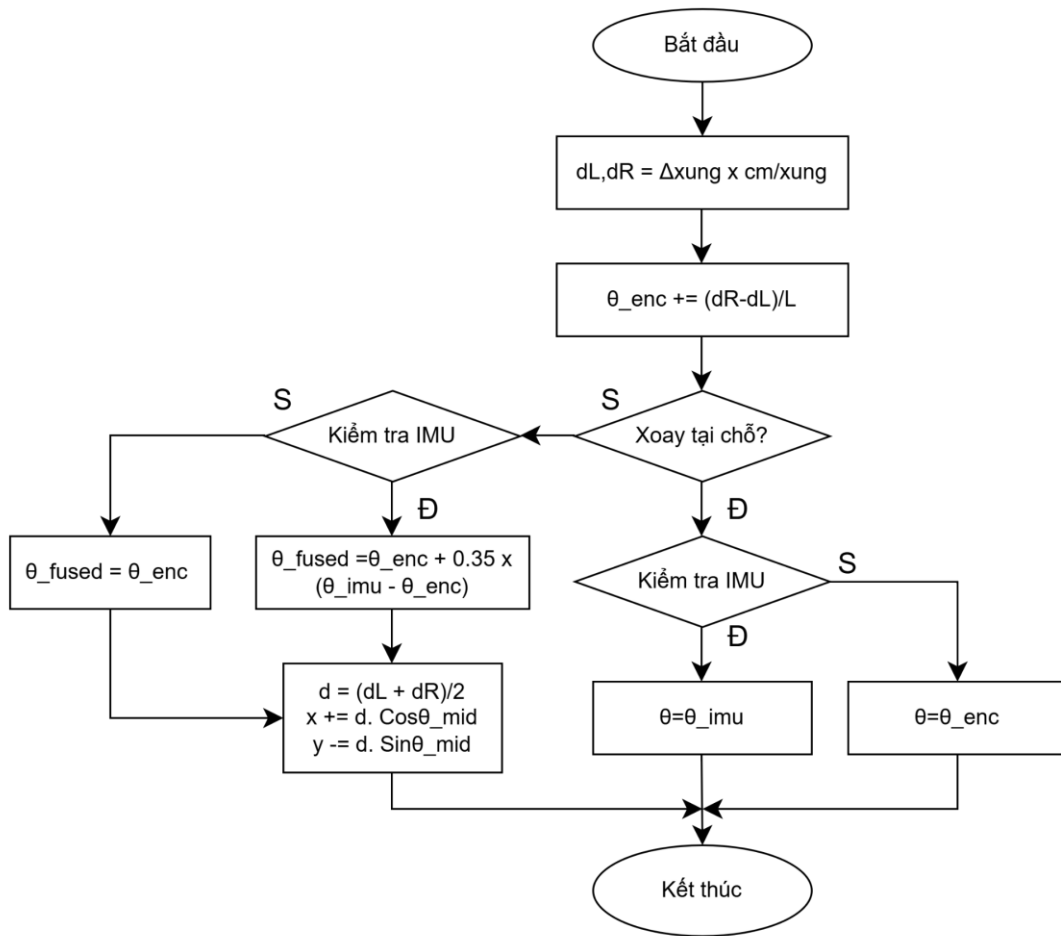
Sau khi khai báo thư viện và khởi tạo (motor, encoder, IMU, WiFi AP, hiệu chuẩn IMU, đặt gốc tọa độ), chương trình vào vòng lặp chính 50 Hz: nhận lệnh UDP → cập nhật IMU → cập nhật odometry → chạy máy trạng thái điều hướng (hoặc thực thi lệnh tay) → gửi telemetry.



Hình 3.11: Lưu đồ giải thuật chính

3.6.2 Lưu đồ chương trình con cập nhật Odometry

Mỗi chu kỳ, tính độ thay đổi xung trái/phải $\rightarrow$ quãng đường dL , $dR \rightarrow$ góc quay encoder. Nếu đang xoay tại chỗ thì chỉ cập nhật hướng (ưu tiên IMU); nếu đi thẳng/cong thì hợp nhất hướng bằng bộ lọc bù rồi cập nhật tọa độ (x, y) .



Hình 3.12: Lưu đồ cập nhật odometry

Các công thức odometry cốt lõi:

Công thức góc quay vi sai: $\Delta\theta_{enc} = \frac{(dR-dL)}{L}$ (3.7)

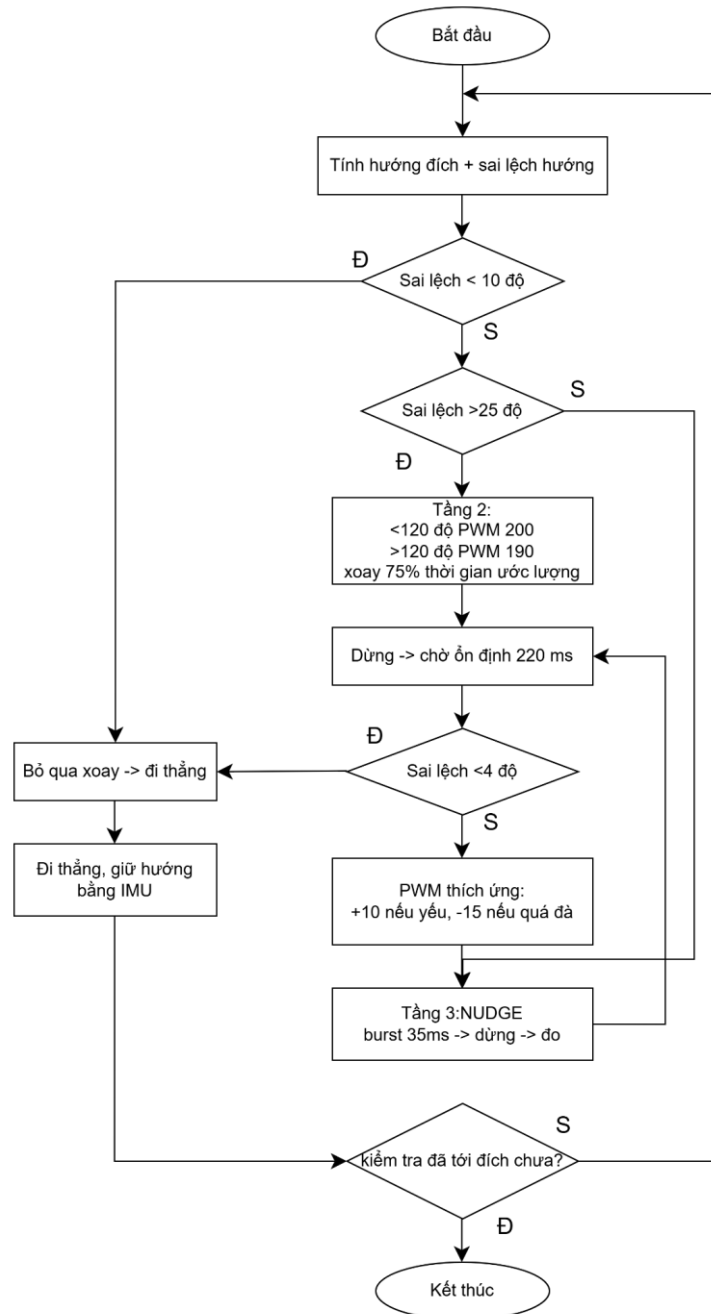
Công thức lọc bù: $\theta_{fused} = \theta_{enc} + 0.35 \times (\theta_{imu} - \theta_{enc})$ (3.8)

Công thức cập nhật vị trí:

$x += d \times \cos(\theta_{mid}); y -= d \times \sin(\theta_{mid})$ (3.9)

3.6.3 Lưu đồ điều hướng node - cơ chế 3 tầng xoay

Đây là phần "trí tuệ" của firmware. Tại mỗi node, navigator tính hướng cần đi và sai lệch hướng, rồi chọn một trong ba chiến lược xoay tùy độ lớn sai lệch, sau đó đi thẳng tới node tiếp theo.



Hình 3.13: Lưu đồ điều hướng node 3 tầng xoay

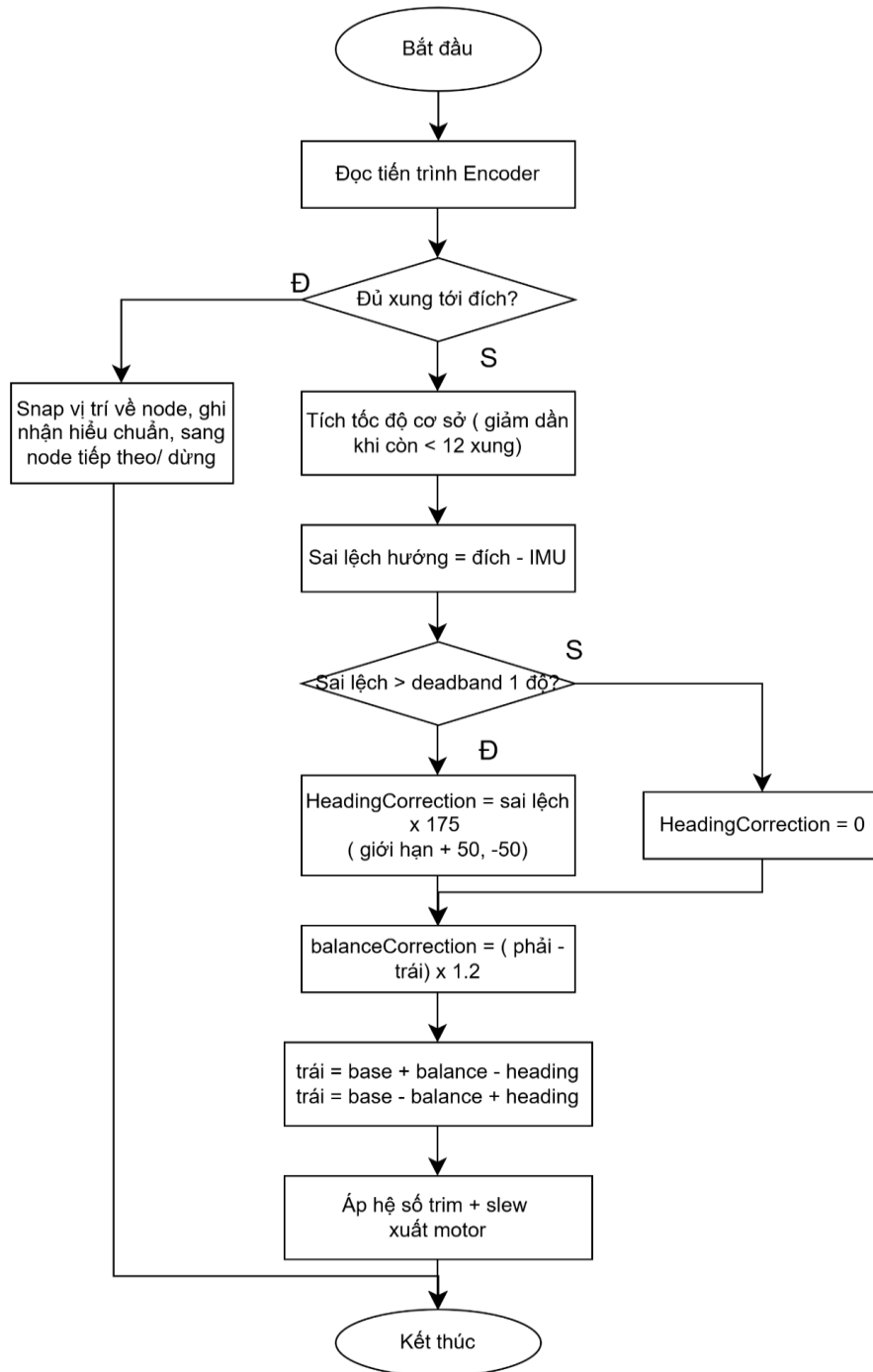
Giải thích ba tầng:

- Tầng 1: Bỏ qua ($< 10^\circ$): sai lệch nhỏ thì không xoay, đi thẳng luôn và để khâu hiệu chỉnh hướng tự kéo lại. Giúp xe chạy mượt, không khựng tại mỗi node.
- Tầng 2: Xoay nhanh ($> 25^\circ$): xoay liên tục ở PWM cao theo thời gian ước lượng ($150^\circ/\text{s} \times 75\%$ để tránh trượt quá), kèm phát hiện vượt đích (cross-target) để dừng ngay khi sai lệch đổi dấu - xử lý đúng trường hợp quay đầu 180° (chống lỗi "angle wrapping").
- Tầng 3: Nudge tinh chỉnh (10° – 25° hoặc phần dư sau xoay nhanh): lặp chu trình cấp xung 35 ms $\rightarrow$ dừng $\rightarrow$ chờ 220 ms cho IMU ổn định $\rightarrow$ đo hướng. PWM tự thích ứng: tăng +10 khi xoay không đủ (thắng ma sát tĩnh), giảm -15 khi xoay quá đà. Robot luôn đứng yên khi đo nên dữ liệu IMU sạch, không bị rung động cơ làm sai. Có timeout 12 giây để báo lỗi nếu kẹt.

Tối ưu node thẳng hàng: nếu node kế tiếp gần như cùng hướng (lệch $< 10^\circ$), xe không dừng mà đi thẳng liên tục qua nhiều node - giảm số lần dừng/xoay, chạy nhanh và mượt hơn.

3.6.4 Lưu đồ chương trình con đi thẳng (heading correction)

Khi đi thẳng, xe đếm xung encoder để biết đã đi được bao xa, đồng thời giữ hướng bằng IMU (không dùng odometry để tránh vòng lặp phản hồi sai). Hai cơ chế hiệu chỉnh chạy song song: hiệu chỉnh hướng (theo IMU) và cân bằng hai bên (theo encoder).



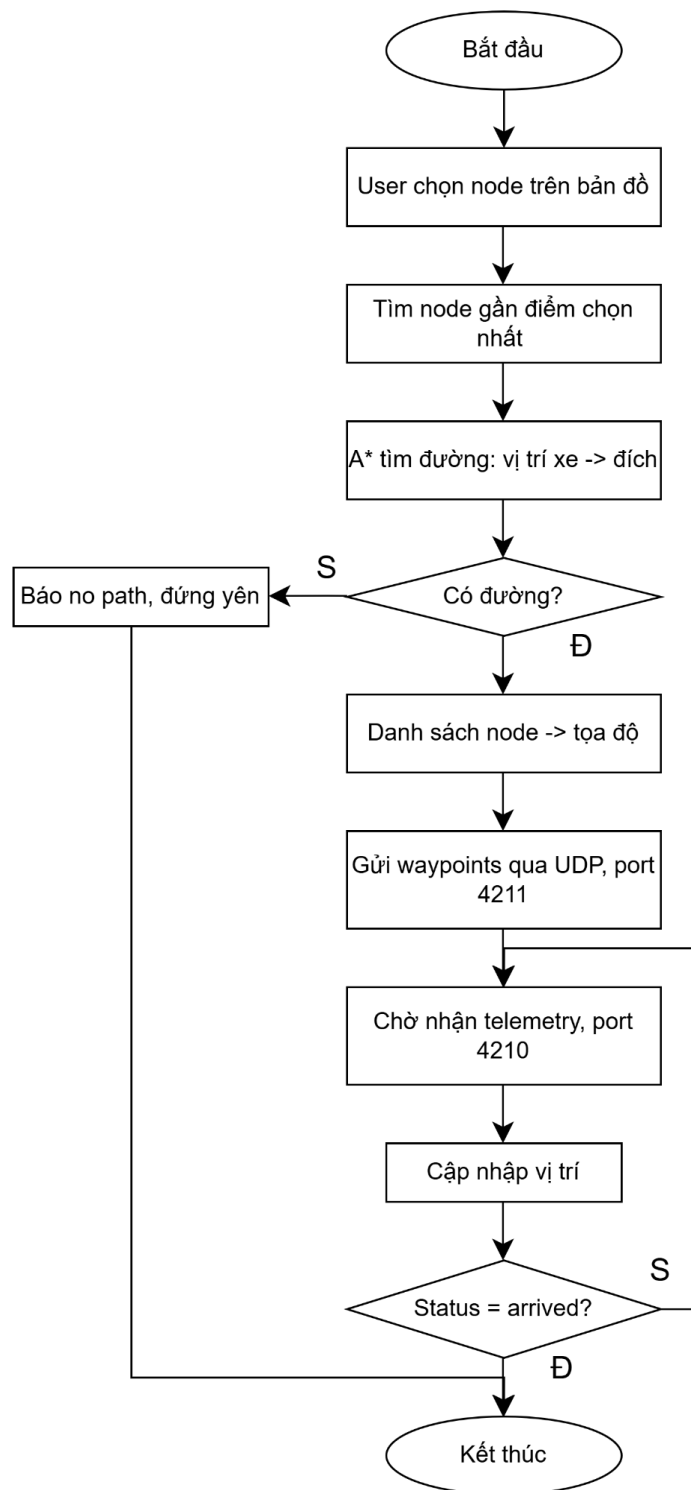
Hình 3.14: Lưu đồ chương trình con đi thẳng

Công thức số xung mục tiêu: $N_{pulse} = \text{round}\left(\frac{dist}{0.5\ m}\right) \times 52$ (3.10)

Công thức 10 hiệu chỉnh hướng:

$$c_{heading} = \text{clamp}[(\theta_{\text{đích}} - \theta_{imu}) \times 175, \pm 50] \quad (3.11)$$

3.6.5 Lưu đồ luồng điều hướng trên ứng dụng (A*)



Hình 3.15: Lưu đồ luồng điều hướng trên app

Cơ chế tự hiệu chuẩn (lồng trong quá trình đi thẳng) - Công thức 3.13, 3.14, 3.15: sau mỗi đoạn thẳng, firmware so sánh số xung hai bên để cập nhật hệ số bù động cơ (motor trim) với tốc độ học 5%/đoạn, đồng thời ước lượng lại tỉ lệ encoder (trung bình trượt 0,94/0,06). Các hệ số được kẹp trong $[0,72 - 1,28]$ và lưu vào NVS flash mỗi 15 giây, giữ lại qua các lần khởi động.

$$trim_{trái} += error \times 0.05; \quad (3.12)$$

$$trim_{phải} -= error \times 0.05; \quad (3.13)$$

$$error = \frac{(xung_{phải} - xung_{trái})}{xung_{trung bình}} \quad (3.14)$$

3.6.6 Lưu đồ giải thuật chế độ tự hành né vật cản

Chế độ tự hành là một máy trạng thái chạy lồng trong vòng lặp 50Hz khi người dùng bấm Start trên ứng dụng. Khối này điều phối ba thành phần: cảm biến HC-SR04 (đo khoảng cách), servo (quét hai bên) và Node Navigator (mượn lại cơ chế xoay 3 tầng đã có để quay $90^\circ/180^\circ$).

Năm trạng thái và luồng chuyển:

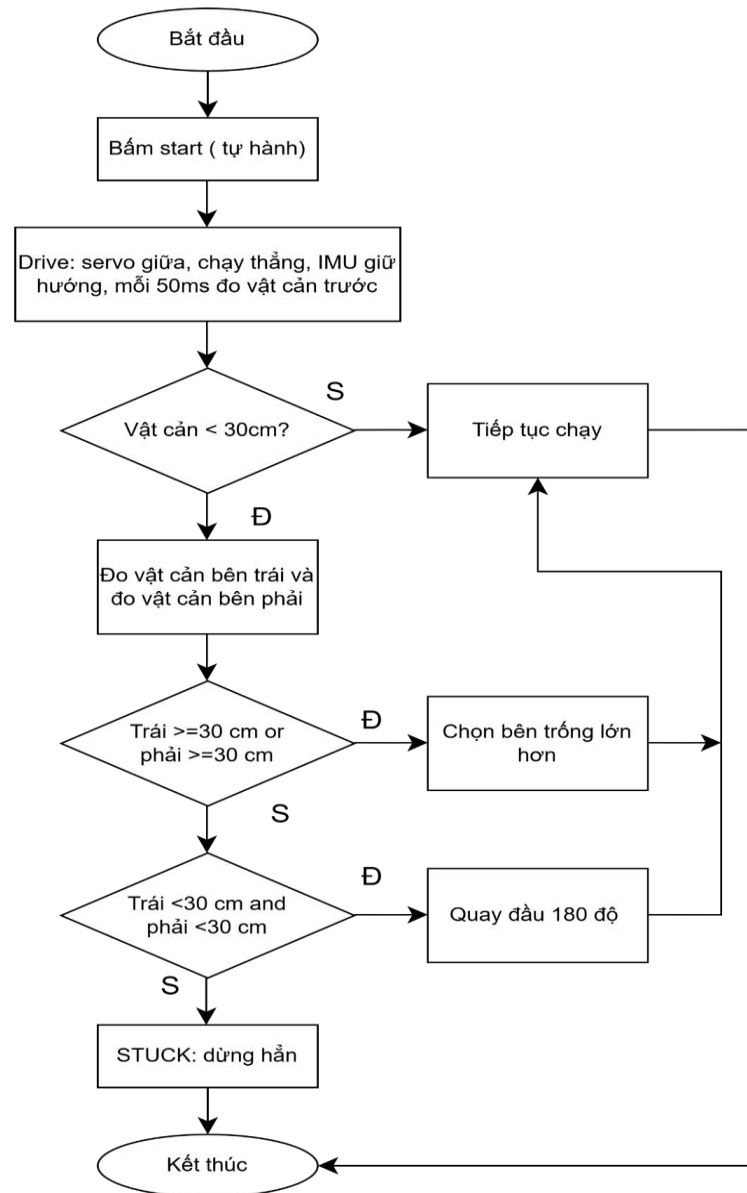
- Drive (Chạy thẳng): servo ở giữa (90°), hai bên chạy tới (PWM 160) đồng thời giữ hướng bằng IMU (mượn công thức hiệu chỉnh hướng của chế độ bám lộ trình). Mỗi $\sim 50ms$ đọc nhanh khoảng cách phía trước.
Nếu
 $< 30cm$ (đọc thêm 1 lần xác nhận để khử nhiễu) $\rightarrow$ dừng, chuyển sang quét.
- ScanLeft (Quét trái): quay servo sang 150° , chờ 350ms cho ổn định rồi đo kỹ (trung vị 5 mẫu) $\rightarrow$ lưu khoảng cách trái.
- ScanRight (Quét phải): quay servo sang 30° , chờ 350ms, đo kỹ $\rightarrow$ lưu khoảng cách phải, đưa servo về giữa rồi ra quyết định.
- Turn (Né): giao mục tiêu hướng cho Node Navigator (180° ; xong thì đồng bộ lại hướng IMU và quay về Drive).
- Stuck (Kẹt): cả ba phía đều hẹp $\rightarrow$ dừng hẳn, báo trạng thái bấm Stop/Start lại.

Quy tắc quyết định sau khi quét:

- Nếu ít nhất một bên $\geq 30cm \rightarrow$ chọn bên có khoảng cách lớn hơn; nếu

bằng nhau thì ưu tiên rẽ trái. Rẽ 90° về bên đó rồi chạy thẳng tiếp.

- Nếu cả hai bên $< 30\text{cm}$ nhưng đều $\geq 25\text{cm}$ $\rightarrow$ quay đầu 180° (đủ chỗ xoay) rồi chạy thẳng.
- Nếu một trong hai bên $< 25\text{cm}$ (không đủ chỗ xoay) $\rightarrow$ dừng hẳn (stuck).



Hình 3.16: Lưu đồ giải thuật chế độ tự hành né vật cản

Ghi chú kỹ thuật:

- Khi đứng yên quét, xe dùng phép đọc kỹ (trung vị 5 mẫu) cho chính xác; khi chạy dùng đọc nhanh để vòng lặp không bị nghẽn (siêu âm có thể chặn tới 30 ms mỗi lần đo khi không có vật phản hồi)
- Pha Turn tái sử dụng toàn bộ cơ chế xoay 3 tầng (xoay nhanh + nudge) của Node Navigator nên góc $90^\circ/180^\circ$ đạt độ chính xác như khi bám lộ trình; sau khi xoay, hướng IMU được đồng bộ lại để các đoạn chạy thẳng kế tiếp không tích lũy sai số.
- Quy ước hướng: rẽ trái = $+90^\circ$ (quay ngược chiều kim đồng hồ, yaw tăng), rẽ phải = -90° ; có thể đảo dấu bằng tham số nếu cách lắp cơ khí ngược chiều.

3.7 DANH SÁCH LINH KIỆN

Bảng 3.8 : Bảng liệt kê các linh kiện của hệ thống

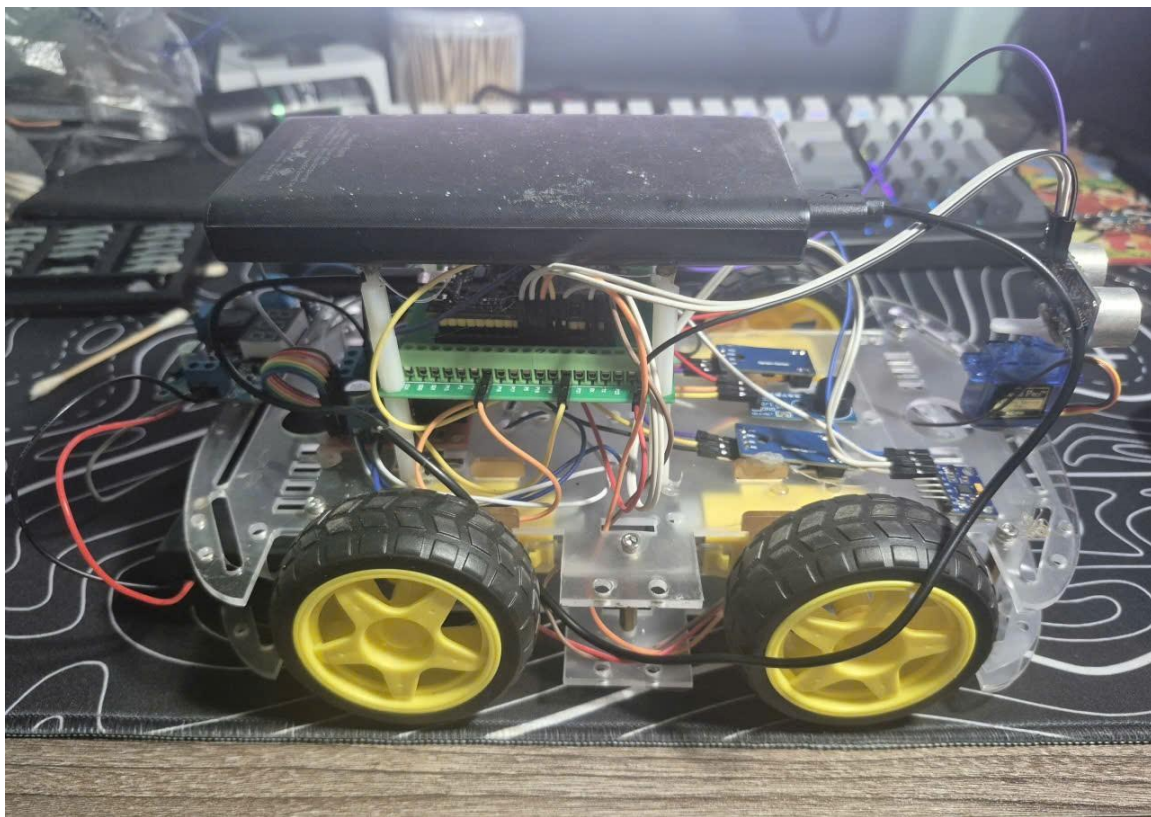
STT	Tên linh kiện	Số lượng
1	Board ESP32 NodeMCU (ESP32-WROOM-32, 38 chân)	1
2	Module mạch lái động cơ L298N (cầu H đôi)	1
3	Động cơ DC giảm tốc (TT motor vàng) + bánh xe	4
4	Encoder quang LM393 + đĩa mã hóa 20 khe	2
5	Cảm biến quán tính MPU-6050 (6 trục)	1
6	Cảm biến siêu âm HC-SR04	1
7	Servo SG90 (180°) quét cảm biến	1
8	Module còi buzzer	1
9	Pin Li-ion 18650 + đế pin (hoặc pin 7.4V)	2
10	Khung xe robot 4 bánh	1
11	Dây nối, jumper, header, ốc vít, ke gắn servo	

3.8 THI CÔNG PHẦN CỨNG VÀ NỐI DÂY

Quá trình thi công phần cứng gồm các bước:

- Lắp khung xe: gắn bốn động cơ DC giảm tốc và bánh xe vào khung (hai bên, mỗi bên hai bánh), lắp đĩa encoder đo tốc độ mỗi bên.
- Gắn mạch: cố định ESP32, L298N và đế pin lên khung; bố trí MPU-6050 ở gần tâm xe (giảm sai số gyro do bán kính); gắn servo SG90 ở mũi xe và lắp HC-SR04 lên cánh tay servo để cảm biến quay theo servo, hướng mặc định nhìn thẳng về phía trước.
- Đi dây theo Bảng 3.1: 6 dây điều khiển ESP32 → L298N; ngõ ra động lực L298N → 4 động cơ (mỗi kênh nối song song 2 động cơ cùng bên) ; 2 dây tín hiệu encoder → GPIO34/32; I2C MPU-6050 → GPIO21/22; Trig/Echo → GPIO5/18; tín hiệu servo → GPIO19; buzzer → GPIO4. Nối chung GND toàn hệ thống.
- Cấp nguồn: pin → động lực L298N; 5V out L298N → VIN ESP32; servo lấy 5V (nên có tụ lọc gần servo để tránh sụt áp gây reset khi servo khởi động).

Lưu ý: Đặt MPU-6050 phẳng, tránh rung; dây encoder và dây động cơ đi tách nhau để giảm nhiễu EMI; cụm servo + HC-SR04 đặt cân bằng ở mũi xe, dây tín hiệu servo đủ chùng để không cản cánh tay quay. Nếu Esp32 bị mất wifi liên tục thì có thể thử nguồn dự phòng khác.



Hình 3.17 Ảnh xe robot sau khi lắp ráp hoàn chỉnh

3.9 LẬP TRÌNH FIRMWARE VÀ ỨNG DỤNG

Firmware (PlatformIO): mã nguồn được tổ chức module hóa theo thư mục communication / controllers / sensors / localization / calibration, biên dịch và nạp bằng PlatformIO:

cd firmware

pio run # biên dịch

pio run --target upload # nạp xuống ESP32

pio device monitor # giám sát Serial

Có môi trường l298n_test riêng để kiểm tra mạch lái động cơ độc lập trước khi tích hợp.

Có môi trường kiểm tra riêng LM293 để kiểm tra xem encoder có bị lệch hay không.

Có môi trường kiểm tra servo và HCSR-04 để kiểm tra xem cảm biến và

servo có hoạt động được hay không.

Ứng dụng (Flutter):

```
cd mobile
```

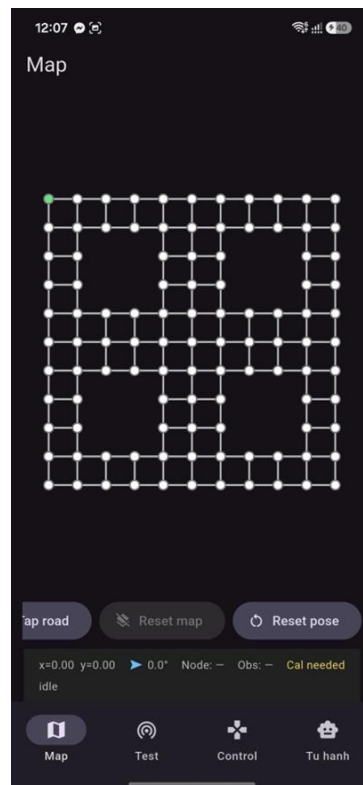
```
flutter pub get
```

```
flutter test # chạy unit test
```

```
flutter run # chạy với UDP thật (cần ESP32)
```

```
flutter run --dart-define=USE_SIMULATOR=true # chạy với giả lập
```

Kiểm thử các chức năng: các module thuật toán quan trọng được kiểm thử tự, chế độ test quay trái phải trên dưới, A* (đường ngắn nhất, không có đường, cạnh bị chặn), nạp bản đồ, bộ dựng waypoint, bộ phân tích telemetry, và toán odometry, chế độ xe tự hành.



Hình 3.18: Giao diện bản đồ trong quá trình xe di chuyển

CHƯƠNG 4: KẾT QUẢ VÀ THẢO LUẬN

4.1 KẾT QUẢ LÝ THUYẾT

Sau khi thực hiện đề tài, về mặt lý thuyết đã đạt được:

- Nắm vững cách sử dụng ESP32 và các chuẩn giao tiếp GPIO, I2C, PWM/LEDC, Trigger-Echo, ngắt ngoài và WiFi/UDP; biết tra cứu và áp dụng datasheet (MPU-6050, L298N, HC-SR04).
- Hiểu và cài đặt được bài toán odometry vi sai và bộ lọc bù kết hợp encoder + IMU; hiểu bản chất sai số dead-reckoning và cách giảm thiểu.
- Cài đặt thành công thuật toán A* với heuristic Euclid và chi phí phạt rẽ; hiểu khái niệm heuristic admissible.
- Thiết kế được giao thức UDP thời gian thực với cơ chế chống trùng, chia mảnh và telemetry.
- Cài đặt được máy trạng thái né vật cản phản ứng (reactive obstacle avoidance) dựa trên cảm biến siêu âm + servo quét, và điều khiển servo bằng LEDC thô (PWM 50 Hz, 16-bit) - hiểu cách phối hợp nhiều thiết bị trong một vòng lặp thời gian thực mà không nghẽn.
- Biết tổ chức một dự án phần mềm module hóa (firmware C++ + app Flutter) kèm kiểm thử đơn vị.

4.2 KẾT QUẢ THỰC HÀNH

Đã hoàn thiện một xe robot tự hành chạy được end-to-end: từ thao tác chạm chọn đích trên app, xe tự tính đường, tự xoay và bám lộ trình qua các node tới đích, với vị trí hiển thị thời gian thực trên bản đồ. Đã thực hiện được quá trình lắp ráp cơ khí, đi dây, nạp firmware, build app, và tinh chỉnh thực nghiệm các tham số (số xung/node, hệ số hiệu chỉnh hướng, PWM xoay, thời gian ổn định nudge). Cơ chế tự hiệu chuẩn giúp xe đi thẳng cân bằng hơn sau vài đoạn. Chế độ giả lập cho phép demo và phát triển app độc lập với phần cứng.

Ngoài ra đã triển khai và chạy được chế độ tự hành né vật cản: xe tự chạy thẳng, gặp vật cản dưới 30 cm thì dừng, quay servo quét hai bên và tự chọn hướng

(rẽ 90° về bên thoáng, quay đầu 180° khi cả hai bên bị chặn nhưng đủ chỗ xoay, hoặc dừng khi kẹt). Người dùng bật/tắt chế độ này bằng nút Start/Stop trên màn hình Tự hành; các trạng thái exploring / scanning / avoiding / stuck và khoảng cách vật cản được phản hồi trực tiếp lên app.

4.2.1 Điều kiện và phương pháp thực nghiệm

Các thí nghiệm được thực hiện trong nhà, trên sàn phẳng (gạch men/xi măng), pin sạc đầy, nguồn IMU và servo tách riêng qua module giảm áp để giảm nhiễu. Dụng cụ đo gồm: thước cuộn (vạch mm) đo khoảng cách, thước đo độ hoặc vòng tròn chia độ in trên giấy đo góc, đồng hồ bấm giây đo thời gian. Số liệu định vị được thu qua telemetry UDP (10 Hz) hoặc Serial Monitor (chu kỳ 500 ms). Các thông số phần cứng liên quan: đường kính bánh 6.5 cm, 20 xung/vòng encoder, khoảng cách hai bánh $b = 10.5 \text{ cm}$, một node $= 0.5 \text{ m} \approx 52 \text{ xung}$ (Công thức 3.11).

4.2.2 Độ chính xác đo quãng đường (odometry)

Phương pháp: Ra lệnh cho xe đi thẳng các quãng 0.5 m, 1.0 m, 2.0 m; đo quãng đường thực tế bằng thước, đồng thời ghi quãng đường do odometry báo về.

Bảng 4.1 Đánh giá độ chính xác ước lượng quãng đường khi xe di chuyển thẳng

Quãng đường lệnh (m)	Odometry báo về (m)	Đo thực tế TB (m)	Sai số TB (cm)	Sai số (%)
0.5	0.5	0.538	3.8	7.6
1.0	1.0	1.063	6.3	6.3
2.0	2.0	1.89	12.3	6.15

Nhận xét: Sai số quãng đường chủ yếu do bánh xe trượt và cách đọc của encoder. Hằng số số xung mỗi node đã được hiệu chỉnh thực nghiệm (49 xung để khớp với tính toán vì sai số được đánh giá là không quá lớn nên không điều chỉnh thêm).

4.2.3 Độ chính xác quay góc IMU

Phương pháp: Đặt xe tại tâm vòng tròn chia độ, ra lệnh xoay 90°, 180°; đọc

góc thực tế đạt được. Cơ chế xoay gồm pha xoay nhanh theo ước lượng thời gian cho góc lớn, sau đó pha nudge (xoay từng nhịp ngắn, đo IMU, lặp) để tinh chỉnh tới khi sai số $\leq$ ngưỡng cho phép 4° .

Bảng 4.2 Kết quả đo độ chính xác xoay góc

Góc lệnh ($^\circ$)	Góc đo TB ($^\circ$)	Sai số TB ($^\circ$)
90	87.3	2.3
180	176.7	3.3

Nhận xét: Việc sử dụng nguồn cho IMU bằng ESP32 giúp làm cho dữ liệu sẽ có một chút sai số khi quay góc lớn ở lần đầu tiên (do vấn đề liên quan đến nhiễu điện của motor), dao động ở pha nudge thì rất ít, hầu như là chính xác. Góc 180° (quay đầu) thường có sai số lớn nhất do quãng xoay dài, quán tính và trượt bánh nhiều hơn. (sai số trung bình $^\circ$, có nằm trong sai số 4° .)

4.2.4 Khả năng giữ hướng khi đi thẳng

Phương pháp: Cho xe đi thẳng 3 m dọc một đường kẻ sẵn; đo độ lệch ngang của xe so với đường chuẩn tại đích và quy đổi ra góc lệch.

Bảng 4.3 Kết quả đánh giá khả năng giữ hướng

Lần đo	Độ lệch ngang tại 3m (cm)	Góc lệch tương đương ($^\circ$)
1	22.1	4.21
2	20.8	3.97
3	21.5	4.09
4	22.3	4.25
5	22.8	4.35
Trung bình	21.9	4.17

Nhận xét: Nhờ vòng giữ hướng kết hợp IMU và cân bằng encoder (Công thức 10, 11), xe bám đường thẳng tương đối tốt. (độ lệch ngang trung bình 21.9cm trên 3 m, tương đương 4.17°).

4.2.5 Độ chính xác điều hướng theo lưới điểm (node)

Phương pháp: Nạp lộ trình gồm nhiều node, cho xe chạy hết lộ trình rồi đo sai số vị trí và sai số góc tại điểm đích so với vị trí lý thuyết.

Bảng 4.4 Kết quả điều hướng theo lộ trình node

Lộ trình	Số node	Tổng quãng đường (m)	Sai số vị trí cuối (cm)	Sai số góc cuối (°)
Đường thẳng 4 node	4	2	17	5
Chữ L	3	1.5	9	6

Nhận xét: Với lộ trình nhiều node thì tính sai số về vị trí sẽ cao lên do độ chính xác của phần cứng là hạn chế.

4.2.6 Độ chính xác cảm biến siêu âm kết hợp servo quét

Phương pháp: Đặt vật cản tại các khoảng cách biết trước (10, 20, 30, 50cm) ở ba hướng quét của servo - trái ($+90^\circ$), giữa (0°), phải (-90°) - và so sánh giá trị HC-SR04 đo được với khoảng cách thực (Công thức 2.1). (Sau khi sửa dải xung servo về 0.5–2.5 ms, servo quét đủ $\pm 90^\circ$ mỗi bên thay vì chỉ $\sim \pm 45^\circ$ như trước, nên phép đo hai bên phản ánh đúng khoảng trống trái/phải.)

Bảng 4.5 Kết quả đo khoảng cách bằng HC-SR04 theo ba hướng quét

Khoảng cách thực (cm)	Đo hướng giữa (cm)	Đo hướng trái $+90^\circ$ (cm)	Đo hướng phải -90° (cm)	Sai số trung bình (cm)
10	10.2	10.4	9.8	0.27
20	20.3	19.6	20.5	0.4
30	29.7	30.6	29.4	0.5
50	50.8	49.1	51.2	0.97

Nhận xét: Độ chính xác này quyết định độ tin cậy của ngưỡng dừng 30cm và ngưỡng quay đầu 25cm trong chế độ tự hành. (sai số đo trung bình 0.535cm trong tầm 10–50cm.

4.2.7 Khả năng né vật cản ở chế độ tự hành

Phương pháp: Cho xe chạy tự hành trong môi trường có vật cản đặt ngẫu nhiên, thực hiện nhiều lượt thử; ghi nhận khoảng cách dừng trước vật cản và việc né có thành công (không va chạm) hay không.

Bảng 4.6 Kết quả thử nghiệm né vật cản tự hành

Lượt thử	Khoảng cách dừng (cm)	Né thành công (Có/Không)	Ghi chú
1	29	Có	né mặt tường
2	28.5	Có	né bàn tay trước mặt
3	29	Có	né mặt tường
4	29	Có	né chân bàn
5	29.2	Có	né mặt tường

Tỉ lệ né thành công: 5 / 5 lượt (100%); khoảng cách dừng lại trung bình:

28.94cm. Trong quá trình di chuyển thì servo sẽ hơi bị giật do tình trạng nhiễu điện của nguồn khi cung cấp cho cả motor và cả servo.

4.2.7 Độ trôi góc hướng của IMU (drift)

Để xe đứng yên hoàn toàn sau khi hiệu chỉnh bias, ghi giá trị góc yaw tại các mốc thời gian để đánh giá độ trôi tích lũy

Bảng 4.7 Kết quả đo của độ trôi góc yaw khi đứng yên

Thời gian đứng yên (s)	Độ trôi raw (°)
45	0
90	0
300	0.1

Nhận xét: Nhờ hiệu chỉnh bias lúc khởi động và vùng chết vận tốc góc (deadband 0.01 rad/s), độ trôi được hạn chế ở mức chấp nhận được trong thời gian một lộ trình điển hình. (trôi 0.1° sau 5 phút.

4.3 NHẬN XÉT VÀ ĐÁNH GIÁ

Về cơ bản hệ thống hoạt động đầy đủ các chức năng chính. Khối điều khiển, định vị và truyền thông hoạt động ổn định. Hạn chế lớn nhất nằm ở sai số định vị tích lũy (dead-reckoning): do chỉ dựa vào odometry tương đối, sau quãng đường dài hoặc nhiều lần xoay, vị trí ước lượng lệch dần so với thực tế (trượt bánh, sai số gyro) - đây là hạn chế cố hữu của phương pháp không có mốc định vị tuyệt đối. Cơ chế nudge step-stop-measure cho góc xoay chính xác nhưng làm thời gian xoay lâu hơn so với xoay liên tục.

Bảng 4.8: Bảng đánh giá hoạt động của hệ thống

STT	Tên khối	Đánh giá hoạt động
1	Khối xử lý trung tâm (ESP32)	Rất tốt
2	Khối động cơ (L298N + DC motor)	Rất tốt
3	Khối encoder (LM393 $\times$ 2)	Khá tốt (đọc do setup phần cứng chỉ cần để lệch sang một chút sẽ làm cho LM293 bị gấp đôi xung đọc được)
4	Khối cảm biến góc IMU (MPU 6050)	Tốt (có trôi nhẹ khi chạy lâu)
5	Khối định vị odometry (lọc bù)	Khá (sai số tích lũy theo quãng đường nếu đi thời gian dài sẽ phá vỡ cấu trúc sai số và từ đó làm cho xe thiếu chính xác)
6	Khối điều hướng A* + Node Navigator	Tốt
7	Khối truyền thông WiFi/UDP	Tốt
8	Khối cảm biến khoảng cách (HC SR04)	Tốt (dùng trong chế độ tự hành; đôi khi nhiễu ở góc/bề mặt hút âm)
9	Khối servo quét + chế độ tự hành	Tốt (né được vật cản; phụ thuộc độ chính

		xác siêu âm)
10	Ứng dụng di động (Flutter)	Tốt

Hệ thống được thiết kế theo định hướng học thuật nên vẫn còn một số hạn chế nhất định. Hiện tại, xe chỉ sử dụng odometry kết hợp IMU để định vị tương đối, chưa hỗ trợ định vị tuyệt đối nên sai số vẫn có thể tích lũy theo thời gian. Chức năng né vật cản được triển khai theo cơ chế phản ứng cục bộ (reactive), cho phép tránh các vật cản xuất hiện phía trước nhưng chưa hỗ trợ lập lại đường đi toàn cục dựa trên bản đồ.

Ngoài ra, cảm biến siêu âm chỉ quan sát theo ba hướng quét nên vẫn tồn tại một số điểm mù trong quá trình phát hiện vật cản. Hệ thống cũng yêu cầu bản đồ được cung cấp trước và chưa có khả năng tự động xây dựng bản đồ môi trường. Do đó, mức độ ứng dụng thực tế hiện chủ yếu phù hợp trong môi trường phòng thí nghiệm và cần được tiếp tục phát triển để đáp ứng các yêu cầu thực tế phức tạp hơn.

4.3.1 Đánh giá mức độ đạt mục tiêu

Đối chiếu trực tiếp với các mục tiêu đề ra ở mục 1.2, kết quả đạt được như sau:

Bảng 4.9 Đánh giá mức độ hoàn thành so với mục tiêu đề ra

Mục tiêu đề ra (mục 1.2)	Mức độ đạt	Minh chứng / căn cứ đánh giá
Tự định vị bằng odometry encoder kết hợp IMU	Đạt	Xe bám lộ trình end-to-end; sai số định lượng đo ở mục 5.2 (Bảng 5.1–5.3)
Tự tìm đường bằng A* và bám lộ trình tới đích người dùng chọn	Đạt	A chạy trên app, xe đi qua các node tới đích; kiểm thử đơn vị A đạt
Firmware ESP32 module hóa (động cơ, cảm biến, odometry, điều hướng)	Đạt	Cấu trúc module hóa + unit test các module thuật toán
Ứng dụng Flutter: bản đồ, A*, telemetry thời gian thực	Đạt	App 4 màn hình, telemetry 10 Hz, có chế độ giả lập
Giao thức UDP độ trễ thấp	Đạt	UDP cổng 4210/4211, chống trùng + chia mảnh lộ trình
Tự hiệu chuẩn giảm sai số định vị	Đạt một phần	Cân bằng đi thẳng cải thiện rõ; sai số tích lũy quãng dài vẫn còn
Né vật cản	Đạt (mức phản ứng cục bộ)	Tự hành tránh được vật trước mặt; chưa lập lại đường toàn cục

4.3.2 Đánh giá chung

Về chức năng, hệ thống đáp ứng đầy đủ các mục tiêu cốt lõi của đề tài (định vị - tìm đường - bám lộ trình - truyền thông - giao diện); các tính năng bắt buộc đều chạy được ổn định, chỉ nhóm tính năng "nâng cao" (tự hiệu chuẩn triệt để, né vật cản toàn cục) đạt ở mức một phần. Theo Bảng 5.8, đa số khối được đánh giá Tốt – Rất tốt; hai khối "Khá/một phần" là định vị odometry (sai số tích lũy) và né vật cản (mới ở mức phản ứng).

Về độ chính xác, hệ thống đạt yêu cầu trong phạm vi phòng thí nghiệm: ở quãng ngắn và ít lần xoay, sai số vị trí-góc nằm trong ngưỡng chấp nhận (số liệu

cụ thể ở mục 5.2); sai số chỉ tăng đáng kể khi quãng đường dài hoặc nhiều lần xoay - đúng bản chất hạn chế của dead-reckoning, và đã được giảm thiểu (không khử hết) bằng bộ lọc bù IMU và tự hiệu chuẩn.

Về độ ổn định và chi phí, các khối điều khiển - truyền thông - ứng dụng hoạt động ổn định, độ trễ thấp, độc lập Internet; toàn hệ thống dùng linh kiện phổ thông giá rẻ, phù hợp mục tiêu nền tảng học thuật chi phí thấp.

Kết luận đánh giá: đề tài hoàn thành các mục tiêu đặt ra; các hạn chế còn lại (định vị tuyệt đối, SLAM, lập lại đường toàn cục) đều nằm ngoài phạm vi nghiên cứu đã giới hạn ở mục 1.4 và được đề xuất ở hướng phát triển (Chương 6).

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 KẾT LUẬN

Đề tài đã thiết kế và xây dựng thành công một xe robot tự hành bốn bánh dẫn động vi sai điều khiển bằng ESP32, chạy được toàn trình từ chọn đích trên ứng dụng đến tự định vị, tự tìm đường và bám lộ trình tới đích. Các kết quả chính đạt được, gắn với thông số cụ thể:

- Tìm đường: người dùng chọn đích trên app, app tự tìm đường ngắn nhất bằng A* trên lưới waypoint 5×5 m (heuristic Euclid + chi phí phạt rẽ).
- Định vị: xe tự định vị bằng odometry kết hợp IMU MPU-6050 qua bộ lọc bù ($\alpha = 0,35$), bám lộ trình bằng cơ chế xoay 3 tầng và giữ hướng khi đi thẳng. Qua đo thực nghiệm (mục 5.2): sai số quãng đường trung bình khoảng 6-7%, sai số xoay góc trung bình 4.17° (ngoài dung sai 4°), độ lệch giữ hướng $\sim 4.17^\circ$ trên 3 m (≈ 22 cm).
- Truyền thông: hiển thị vị trí thời gian thực trên bản đồ qua UDP (telemetry 10 Hz, vòng điều khiển 50 Hz), độ trễ thấp, hoạt động độc lập không cần Internet.
- Né vật cản & tiện ích: chế độ tự hành né vật cản dùng HC-SR04 + servo quét (ngưỡng dừng 30 cm; tỉ lệ phát hiện vật cản 100% trong thử nghiệm có kiểm soát) để tự dừng, dò hai bên và chọn hướng; hỗ trợ điều khiển tay, tự hiệu chuẩn (lưu NVS) và chế độ giả lập để kiểm thử.

Hệ thống có kiến trúc module hóa rõ ràng, chi phí thấp, hoạt động ổn định ở quy mô phòng thí nghiệm, đáp ứng đầy đủ các mục tiêu đề ra và là nền tảng tốt để học tập, phát triển tiếp.

5.2 HƯỚNG PHÁT TRIỂN

Để nâng cao độ chính xác và khả năng ứng dụng thực tế, đề tài có thể phát triển theo các hướng:

- Bổ sung định vị tuyệt đối để khắc phục sai số tích lũy: thêm cảm biến

mốc (line/landmark), camera nhận diện thẻ ArUco, hoặc UWB anchor.

- Nâng cấp né vật cản từ phản ứng lên có bản đồ: kết hợp chế độ tự hành hiện có với bản đồ - khi gặp vật cản thì chặn cạnh đường tương ứng và cho A* tự tính lại đường (replan) thay vì chỉ tránh cục bộ; thay/thêm cảm biến ToF/LiDAR mini hoặc nhiều siêu âm để giảm điểm mù và quét nhanh hơn (thay cho việc xoay servo từng bên).
- Nâng cấp định vị bằng SLAM để xe tự xây bản đồ thay vì dùng bản đồ biết trước.
- Cải tiến điều khiển: áp dụng bộ điều khiển PID/đóng vòng vận tốc bằng encoder cho động cơ; xoay liên tục có phản hồi thay cho nudge để rút ngắn thời gian.
- Mở rộng hệ thống IoT: đưa dữ liệu lên đám mây, điều khiển/giám sát từ xa, phối hợp nhiều robot thành một mạng lưới.

PHỤ LỤC

Mã nguồn hệ thống (firmware ESP32 + ứng dụng Flutter):
https://github.com/ngxziet/doantotnghiep_viet

Cấu trúc mã nguồn:

- firmware/src/ - firmware ESP32 (PlatformIO): communication/ (UDP), controllers/ (motor, node navigator, autonomous explorer), sensors/ (encoder, MPU-6050, HC-SR04, servo quét), localization/ (odometry), calibration/ (tự hiệu chuẩn), config.h, main.cpp.
- mobile/lib/ - ứng dụng Flutter: models/, planning/ (A*, road map, waypoint builder), services/ (UDP, simulator), screens/ (4 màn hình: Map, Test, Manual, Tự hành).
- map_data/environment.json -bản đồ lưới waypoint 5×5 m.
- docs/ - tài liệu (codebase summary, hướng dẫn thuật toán firmware).

Thông số mạng ESP32: SSID RobotCar · Mật khẩu 12345678 · IP 192.168.4.1 · Cổng telemetry 4210 · Cổng lệnh 421

TÀI LIỆU THAM KHẢO

- [1] Siegwart, R.; Nourbakhsh, I. R.; Scaramuzza, D., Introduction to Autonomous Mobile Robots, 2nd ed., 2011.
- [2] Fragapane, G.; Ivanov, D.; Peron, M.; Sgarbossa, F.; Strandhagen, J. O., “Increasing flexibility and productivity in Industry 4.0 production networks with autonomous mobile robots and smart intralogistics,” 2021.
- [3] Borenstein, J.; Everett, H. R.; Feng, L., Where am I? Sensors and Methods for Mobile Robot Positioning, 1996.
- [4] Cadena, C. et al., “Past, Present, and Future of Simultaneous Localization and Mapping: Toward the Robust-Perception Age,” 2016.
- [5] Al Khatib, E. I. et al., “Multiple Sensor Fusion for Mobile Robot Localization and Navigation Using the Extended Kalman Filter,” 2015.
- [6] Ly, T. T. K. et al., “Autonomous Mobile Robot Localization by Using IMU and Encoder Data Fusion Technique by Kalman Filter,” 2024.
- [7] Housein, A. A. et al., “Extended Kalman Filter Sensor Fusion in Practice for Mobile Robot Localization,” 2022.
- [8] Huang, G. et al., “Multi-Sensor Fusion for Wheel-Inertial-Visual Systems Using a Fuzzification-Assisted Iterated Error State Kalman Filter,” 2024.
- [9] Mahony, R.; Hamel, T.; Pflimlin, J.-M., “Nonlinear Complementary Filters on the Special Orthogonal Group,” 2008.
- [10] Dijkstra, E. W., “A note on two problems in connexion with graphs,” 1959.
- [11] Hart, P. E.; Nilsson, N. J.; Raphael, B., “A Formal Basis for the Heuristic Determination of Minimum Cost Paths,” 1968.
- [12] LaValle, S. M., Planning Algorithms, 2006.
- [13] Shahapur, V.; Dixon, B.; Bharti, U., “Robotic Path Planning Implementation using Search Algorithms,” 2024.
- [14] Yao, M. et al., “Global Path Planning for Differential Drive Mobile Robots Based on Improved BSGA Algorithm,” 2023.
- [15] Yasin, J. N. et al., “Low-cost ultrasonic based object detection and collision avoidance method for autonomous robots,” 2020.
- [16] Maier, A.; Sharp, A.; Vagapov, Y., “Comparative analysis and practical implementation of the ESP32 microcontroller module for the internet of things,” 2017.
- [17] Google, “Flutter Documentation và Dart Language Tour,” [Trực tuyến]. Available: <https://docs.flutter.dev/>.
- [18] Espressif Systems, “ESP32 Technical Reference Manual,” 2023. [Trực tuyến]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf.
- [19] NXP Semiconductors, “I2C-bus Specification and User Manual (UM10204),” 2021. [Trực tuyến].

- [20] Hart, D. W., Power Electronics, 2011.
- [21] Espressif Systems, “ESP-IDF / Arduino-ESP32 Programming Guide (LEDC, GPIO interrupts, WiFi SoftAP),” [Trực tuyến]. Available: <https://docs.espressif.com/>.
- [22] Texas Instruments, “LMx93, LM2903 Differential Comparators Datasheet,” 2022. [Trực tuyến]. Available: <https://www.ti.com/lit/ds/symlink/lm393.pdf>.
- [23] IEEE, IEEE Std 802.11-2020, 2021.
- [24] Postel, J., “User Datagram Protocol RFC 768,” 1980. [Trực tuyến]. Available: <https://www.rfc-editor.org/rfc/rfc768>.
- [25] Lynch, K. M.; Park, F. C., Modern Robotics: Mechanics, Planning, and Control, 2017.
- [26] Corke, P., Robotics, Vision and Control: Fundamental Algorithms in MATLAB, 2nd ed., 2017.
- [27] Thrun, S.; Burgard, W.; Fox, D., Probabilistic Robotics, 2005.
- [28] Dudek, G.; Jenkin, M., Computational Principles of Mobile Robotics, 2nd ed., 2010.
- [29] Kalman, R. E., “A New Approach to Linear Filtering and Prediction Problems,” 1960.
- [30] Welch, G.; Bishop, G., “An Introduction to the Kalman Filter,” 2006.
- [31] Russell, S. J.; Norvig, P., Artificial Intelligence: A Modern Approach, 4th ed., 2021.
- [32] Koenig, S.; Likhachev, M., “D Lite,” 2002.
- [33] PlatformIO Labs, “PlatformIO Documentation,” [Trực tuyến]. Available: <https://docs.platformio.org/>.
- [34] Espressif Systems, “ESP32 Series Datasheet,” 2024. [Trực tuyến]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf.
- [35] Espressif Systems, “ESP32-WROOM-32 Datasheet,” 2024. [Trực tuyến]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32_datasheet_en.pdf.
- [36] STMicroelectronics, “L298 Dual Full-Bridge Driver Datasheet,” [Trực tuyến]. Available: <https://www.st.com/resource/en/datasheet/l298.pdf>.
- [37] InvenSense (TDK), “MPU-6000/MPU-6050 Product Specification & Register Map,” 2024. [Trực tuyến]. Available: <https://invensense.tdk.com/products/motion-tracking/6-axis/mpu-6050/>.
- [38] SparkFun Electronics, “HC-SR04 Ultrasonic Sensor Module,” [Trực tuyến]. Available: <https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf>.
- [39] Đ. D. N. A. N. V. H. P. T. T. Nguyễn Minh Đăng, Thiết kế và thi công Robot quét và vẽ bản đồ địa hình 2D, Trường Đại học Sư phạm Kỹ thuật Thành phố Hồ Chí Minh, 2025.